



香港科技大學(廣州)
香港
科技大学 (广州)



香港
香港科学大学
和技术

AIAA 2205： 人工智能导论

戴恩艳

香港科技大学 (广州)

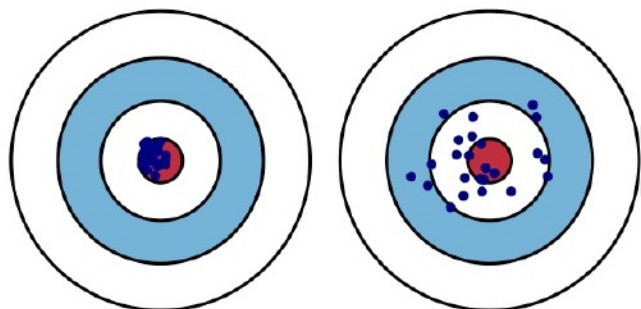
《最后的演讲回忆》

偏差和方差

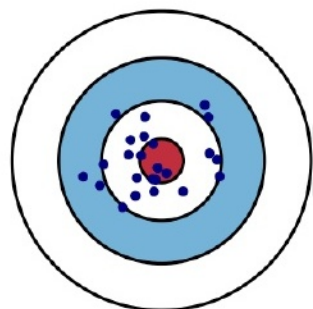
☒ 偏差：模型的**平均预测值**与我们试图预测的**正确值**之间的差异。

☒ 方差：方差大的模型非常关注**训练数据**，但在**测试数据**上泛化能力差。

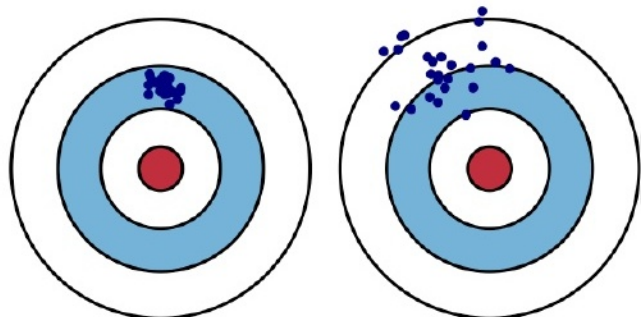
低偏差和
低方差



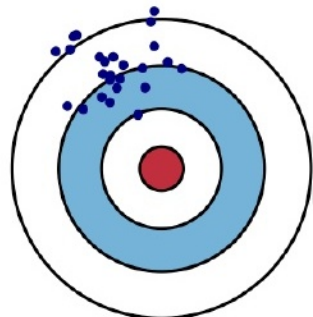
低偏差和高
方差



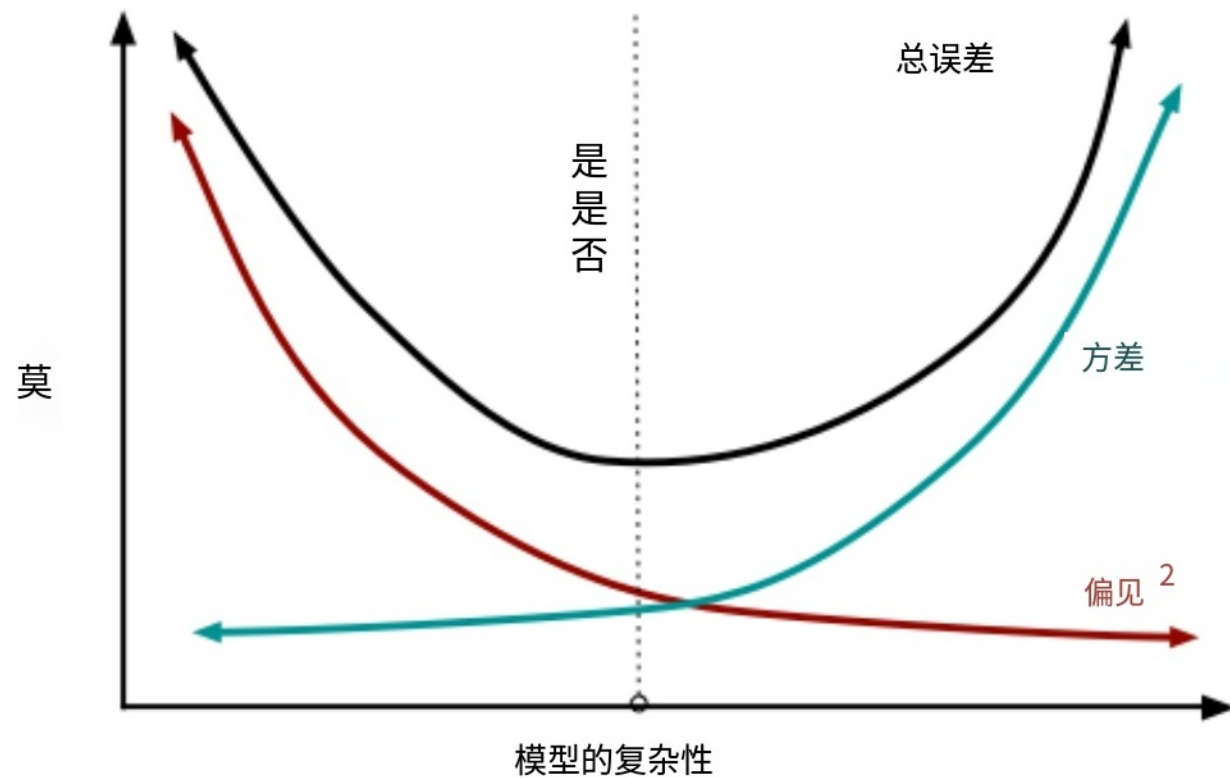
大偏差和低
方差



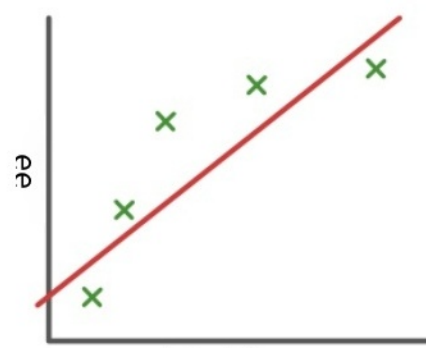
大的偏差和
大的方差



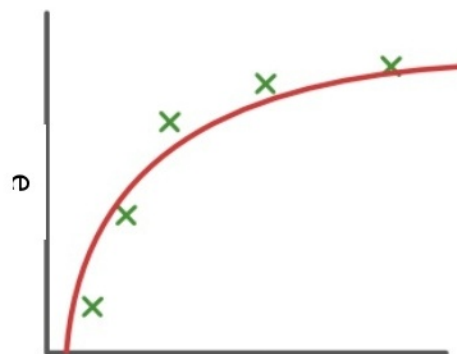
偏差 - 方差图



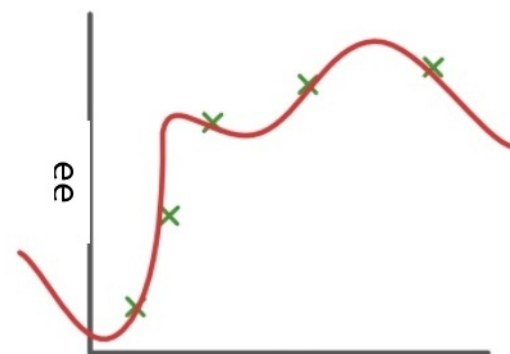
过拟合和欠拟合



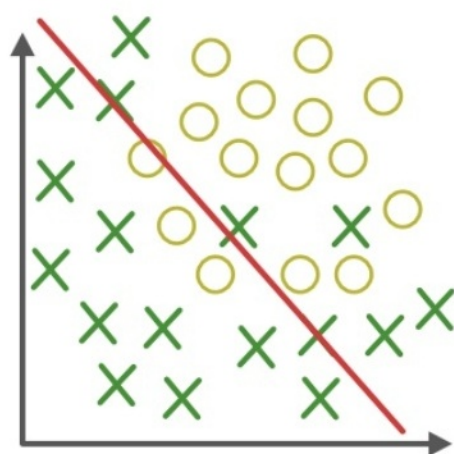
尺寸 θ +
 $\theta_0 + \theta_1 x$



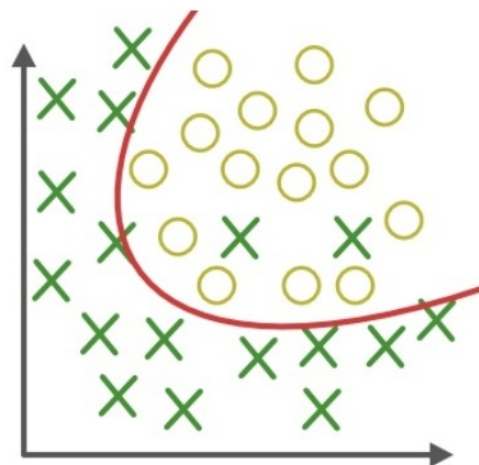
大小
 $\theta_0 + \theta_1 x + \theta_2 x^2$



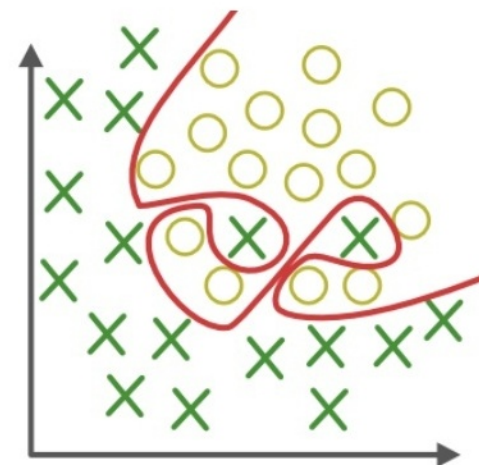
大小
 $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$



不合身 (太简单)
解释方差)



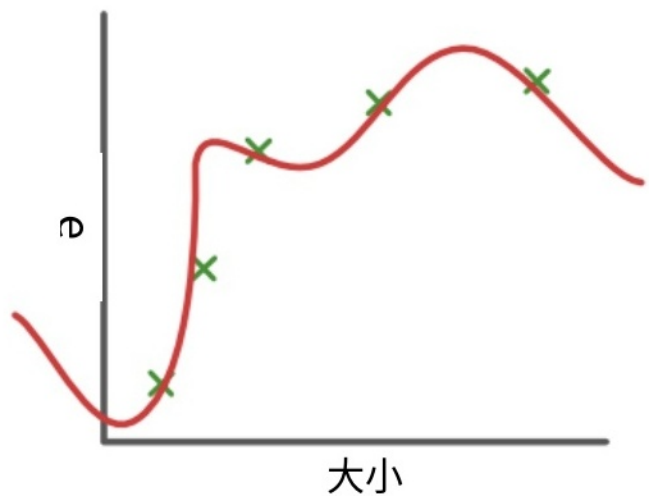
Appropriate-fitting



过度学习
(强制拟合——好得
令人难以置信)

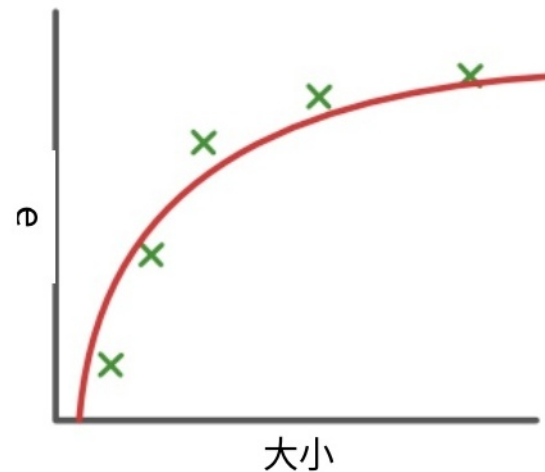
DG

正则化的直觉



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

如果我们限制 θ_3 和 θ_4 的值很小



$$\approx \theta_0 + \theta_1 x + \theta_2 x^2$$

常见正则化项

我们如何鼓励较小的权重？或者惩罚较大的权重？

LASSO 回归器：对权重绝对值之和进行惩罚（L1 范数）

$$\arg \min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |w_i|$$

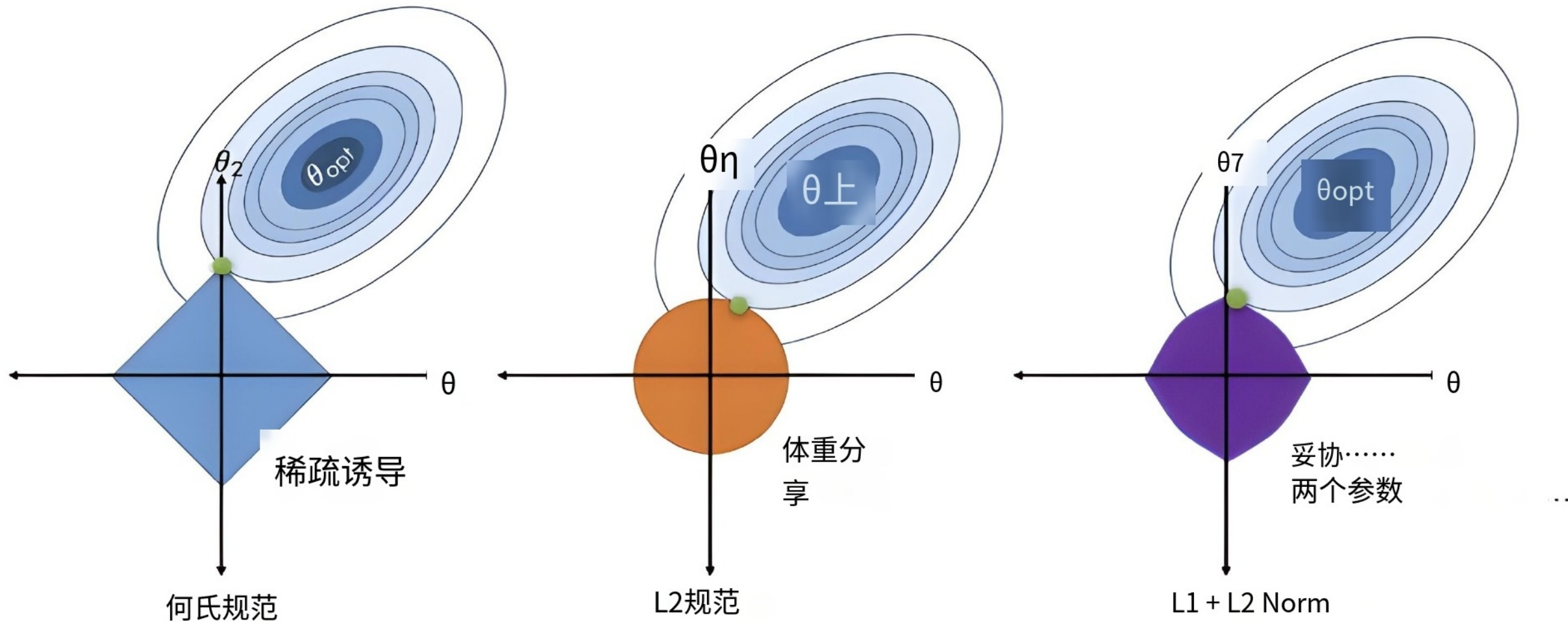
岭回归器：对权重平方和进行惩罚（L2 范数）

$$\arg \min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |w_i|^2$$

λ ：需手动调整的超参数

LASSO 导致稀疏性

L1 正则化强制产生稀疏解：
损失轮廓首先触及 L1 正则化项的尖锐拐角。



评估指标与调优

混淆矩阵

		Predicted Class	
		Yes	No
Actual Class	Yes	TP	FN
	No	FP	TN

$$\text{Accuracy} = \frac{TP+TN}{n}$$

- TP: 真阳性
- FP: 误报
- FN: 假阴性
- TN: 真阴性

分类指标

$$\text{Accuracy} = \frac{TP + TN}{n}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

实际阳性中被预测为阳性的比例

预测为阳性的样本中实际为阳性的比例

		预测类	
		是	否
D	是	胡志明市	FN
	否	《外交政策》	TN

精度 = $\frac{TP}{TP + FP}$

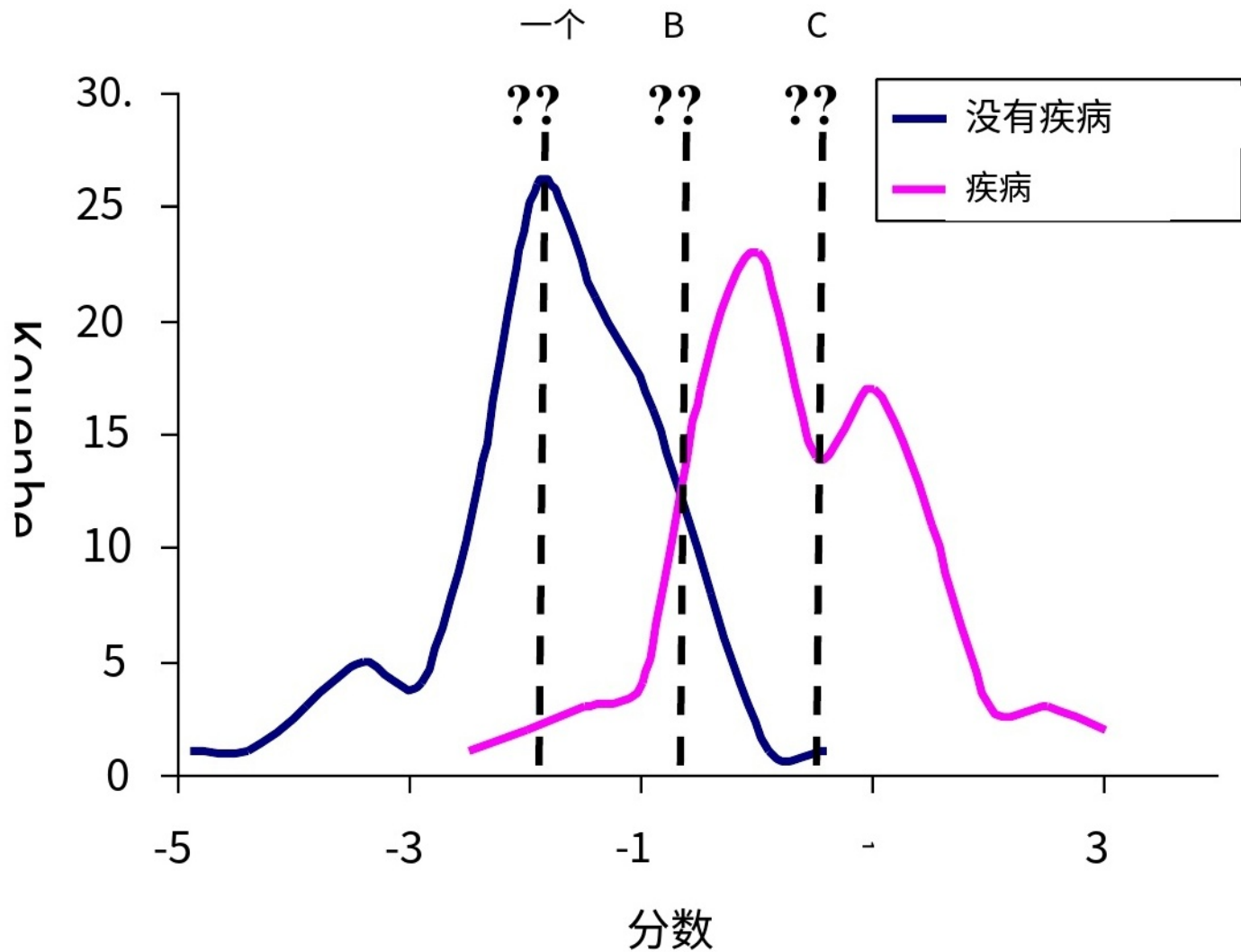
分类指标

F1 分数结合了准确率和召回率

$$\text{F1 Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

视觉视角

正类别的阈值可以进行调整



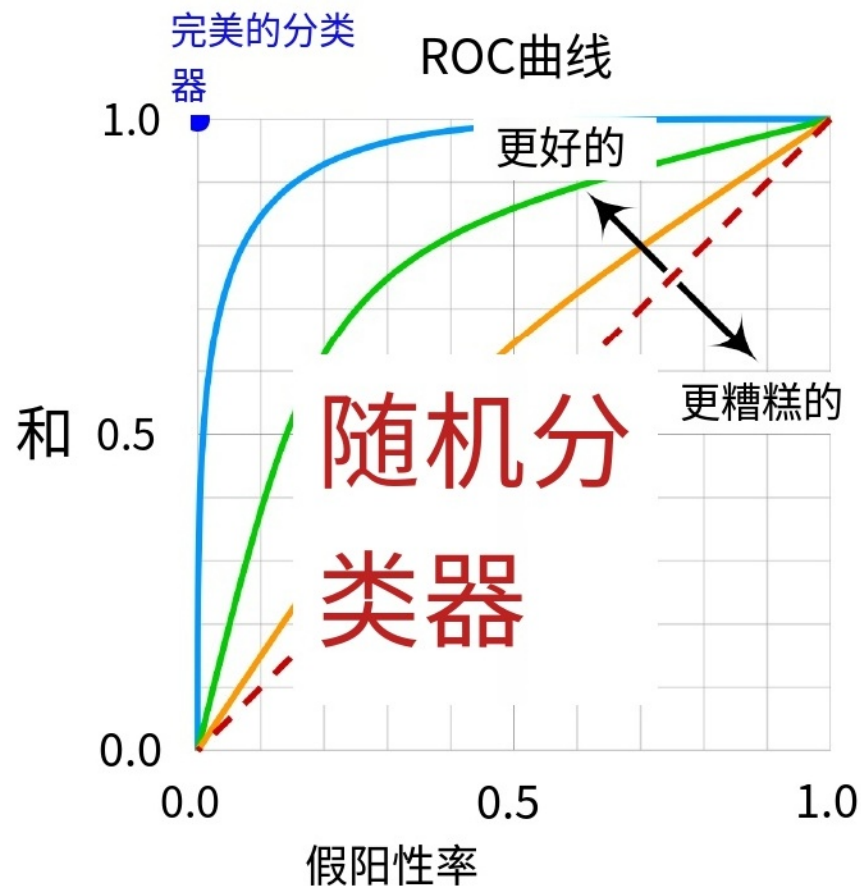
哪一个是最好的?



扫描此码以作答

ROC 曲线

调整正类的阈值可得到以下的 ROC 曲线



$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

ROC-AUC 分数: ROC 曲线下面积

示例

问题与数据（乳腺癌）

诊断结果	手术病理结果		总计
	患者 (P)	非患者 (N)	
积极的	80	100	180
否定的	20	800	820
总计	100	900	1000

计算真正例率和假正例率。



扫描此码以作答

摘要

如果诊断测试的**真阳性率**相对较低，就会有假阴性患者。
这会延误患者的就诊，影响疾病的进展和康复，甚至导致患者过早死亡。

如果诊断测试的**假阳性率**相对较高，就会出现很多假阳性患者。
这会浪费医疗资源，并导致患者无端恐慌和焦虑。

超参数调优与模型选择

给定数据：

训练数据 Z_{train}

测试数据 Z_{test}

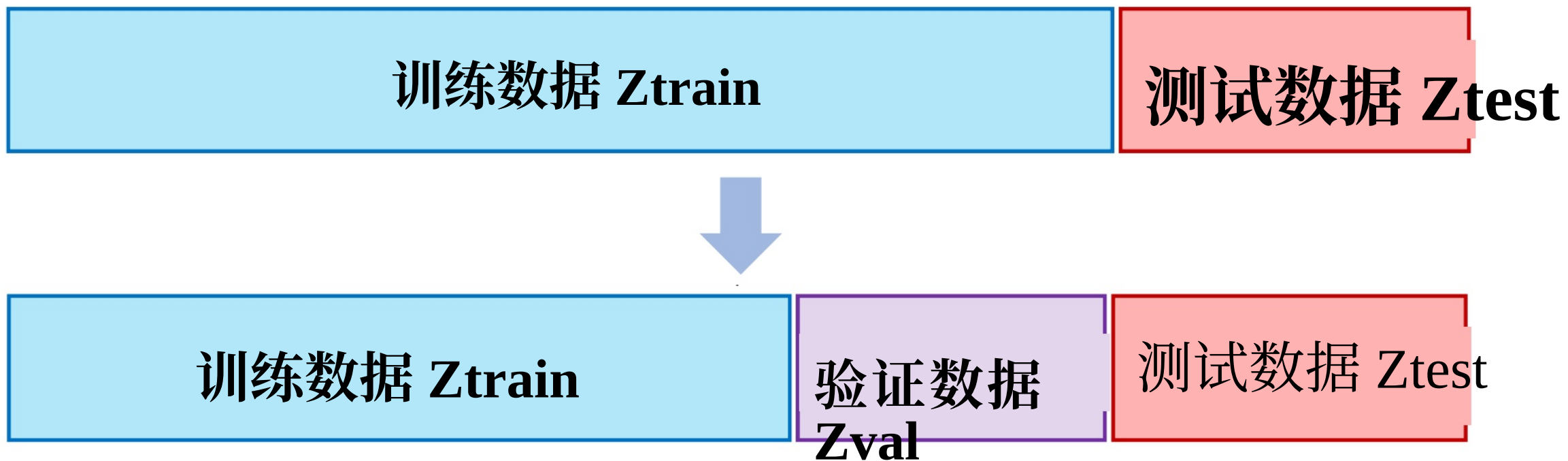
目标：选择具有最佳泛化能力的超参数/模型。

我们能否直接选择使测试集 Z_{test} 最小化的超参数？

不！！这会导致在测试集上过拟合。

- 这会泄露测试数据的信息
- 测试数据的结果并不能准确反映模型的泛化能力。

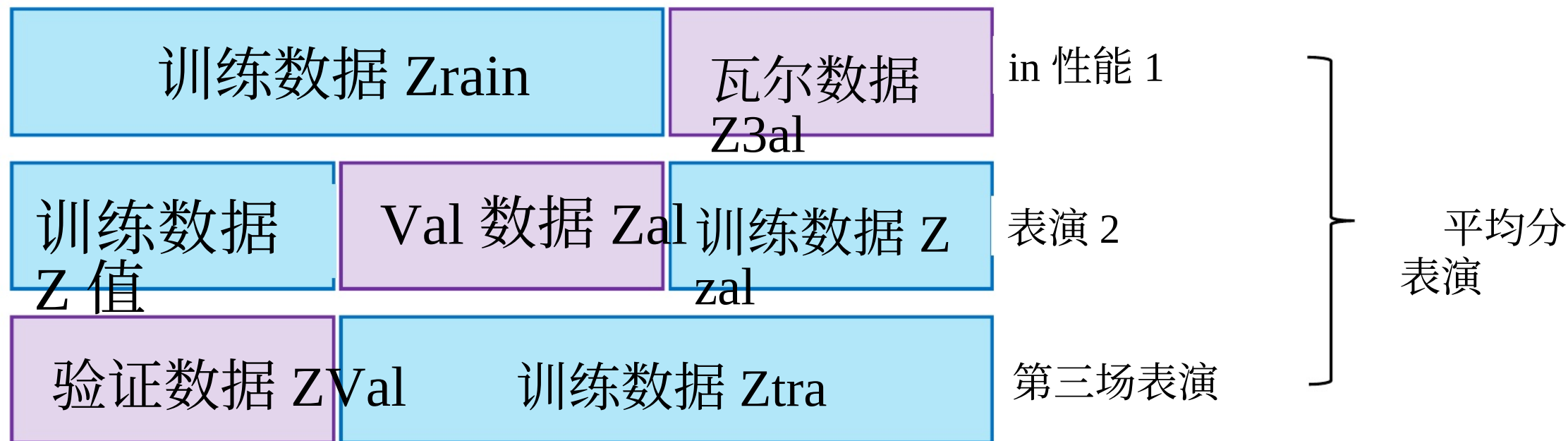
训练/验证/测试集划分



解决方案：在预留的验证数据上调整超参数/模型

K 折交叉验证

三重的一个例子



根据平均性能调整模型 避免在选定的验证集上过拟合

一个例子

假设我们有一个包含 6 个观测值的数据样本：

1 [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]

- 第一步是选择一个 k 值，以确定用于拆分数数据的折数。
- 在这里，我们将使用 $k = 3$ 的值。这意味着我们将打乱数据，然后将其分成 3 组。
- 因为我们有 6 个观测值，所以每个组将有相等的 2 个观测值。

一个例子

例如：

1 折叠左：[0.5, 0.2]

2 折叠 2：[0.1, 0.3]

3 折叠 3：[0.4, 0.6]

训练并评估了三个模型，每次都将其中的一个折算作保留的测试集。

例如：

- 模型 1：在 Fold1 和 Fold2 上训练，在 Fold3 上测试
- 模型 2：在 Fold2 和 Fold3 上训练，在 Fold1 上测试
- 模型 3：在折合 1 和折合 3 上进行训练，在折合 2 上进行测试

将这三个模型的平均值以及标准差作为最终结果。

1. 研究背景与意义：随着人工智能技术的飞速发展，AI在各行各业的应用日益广泛。然而，AI技术的深度研究和探索仍然面临诸多挑战，需要专业的研究人员进行深入探索。

2. 研究目标与任务：本研究旨在探索AI技术的最新进展，分析其在各个领域的应用现状，并提出具有前瞻性的研究方向和任务。

3. 研究方法与技术路线：本研究将采用文献调研、实验验证、案例分析等多种方法，结合最新的AI技术，进行深入的研究和探索。

上节课回顾

分类指标

		预测类	
		是	否
实际类	是	胡志明市	FN
	否	《外交政策》	TN

精度 = $\frac{TP}{TP + fp}$

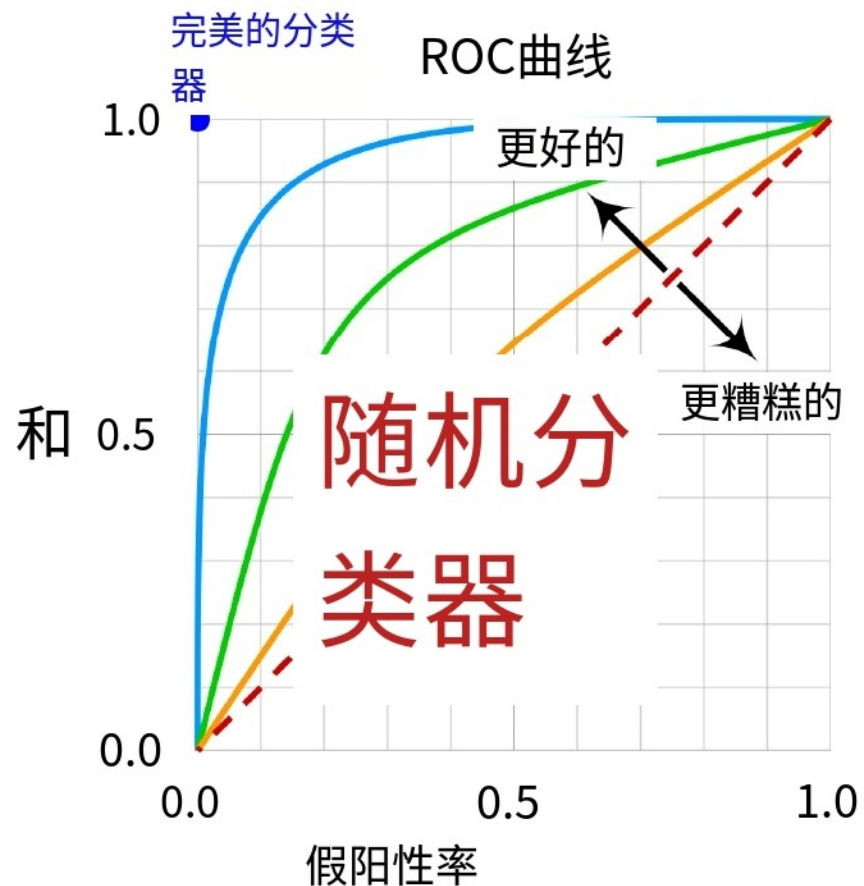
$$\text{Accuracy} = \frac{TP + TN}{n}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

- 实际阳性中被预测为阳性的比例
- 预测为阳性的样本中实际为阳性的比例

ROC 曲线

调整正类的阈值可得到以下的 ROC 曲线



$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

ROC-AUC 分数: ROC 曲线下面积

训练/验证/测试集划分

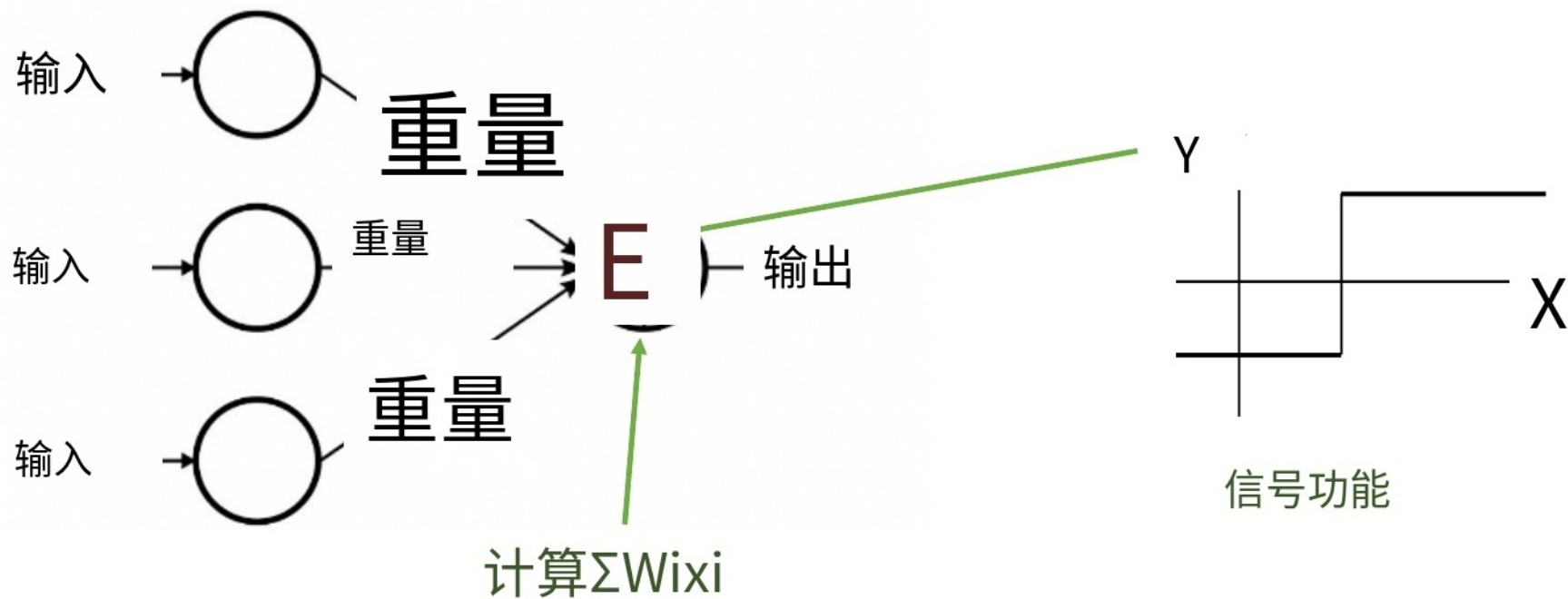


解决方案：在预留的验证数据上调整超参数/模型

多层感知器

感知

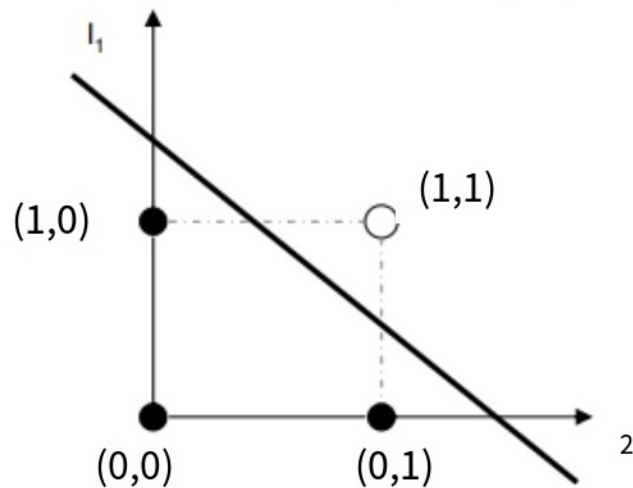
- 感知器模拟 1 个神经元
- 方程: $Y = \text{sign } \sum(W_i X_i)$



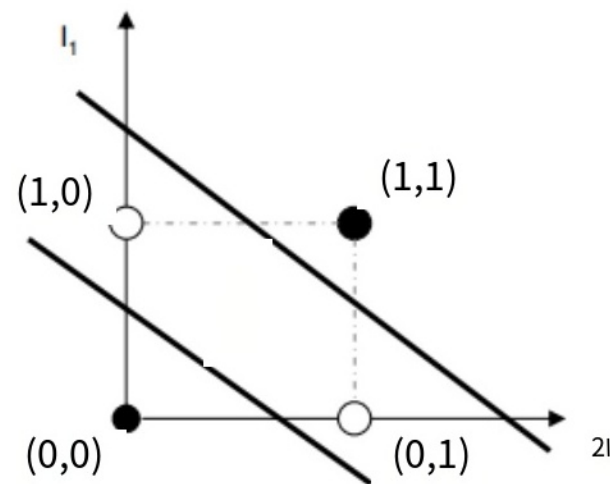
感知的异或问题

感知无法解决异或问题

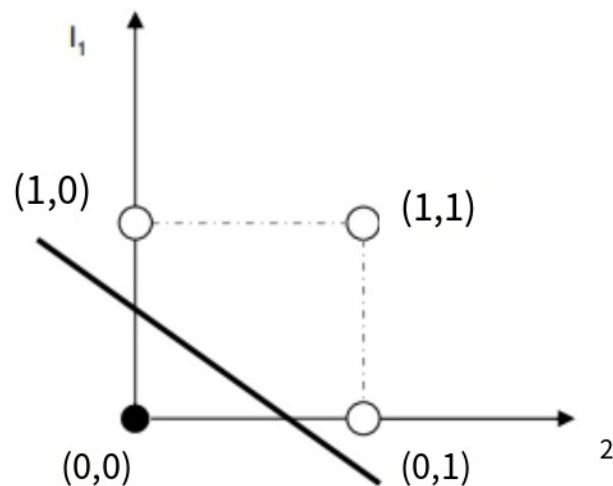
和		
x	12	出
0	0	0
0	1	0
1	0	0
1	1	1



异或		
I_1	2	出
0	0	0
0	1	1
1	0	1
1	1	0



或		
I_1	2	出
0	0	0
0	1	1
1	0	1
1	1	1

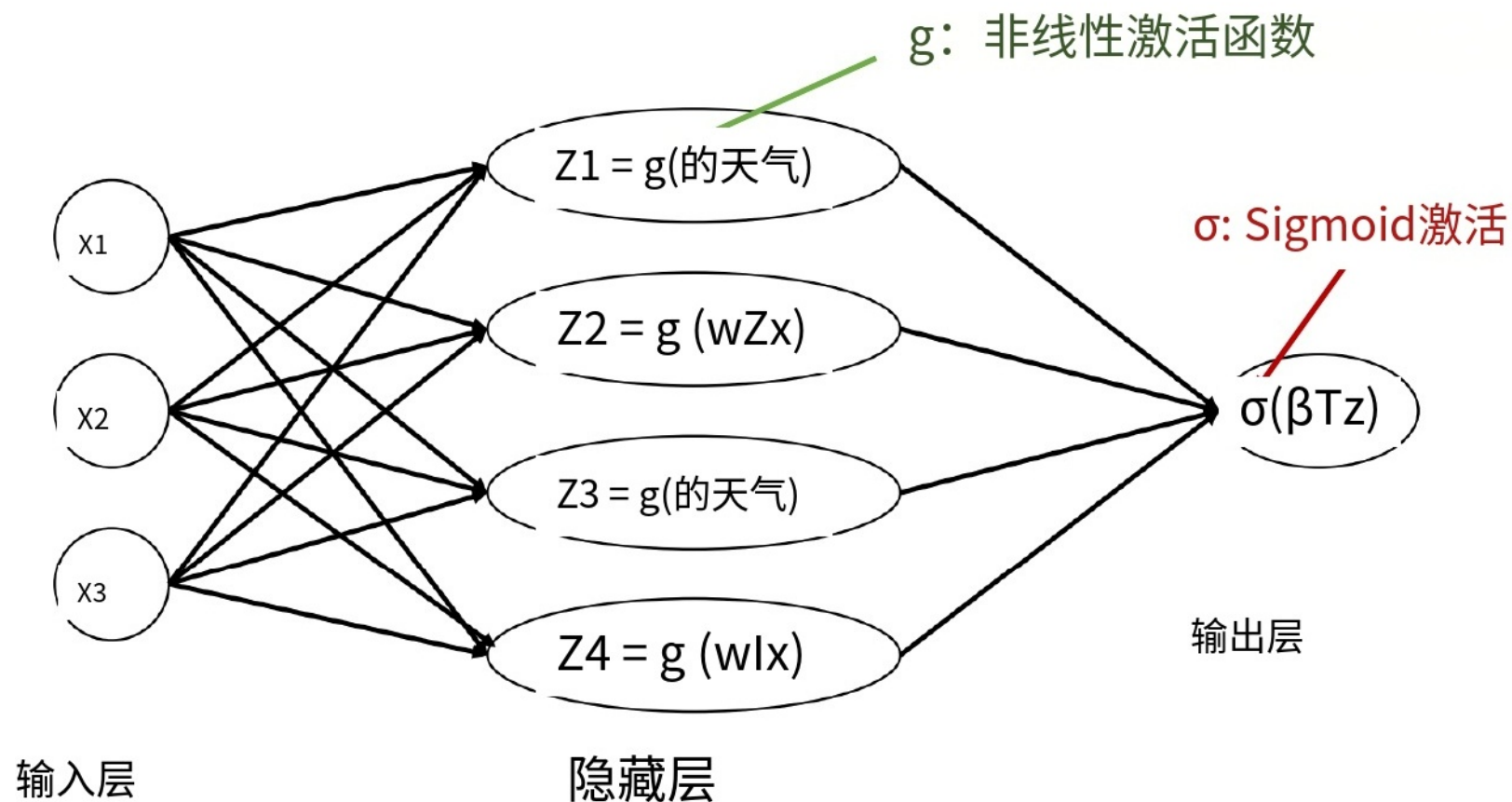


多层感知器

多层感知机由多层神经网络组成

对于二分类问题: $F(X) = \sigma(\beta^T G(WX))$

多层感知机可以堆叠许多隐藏层

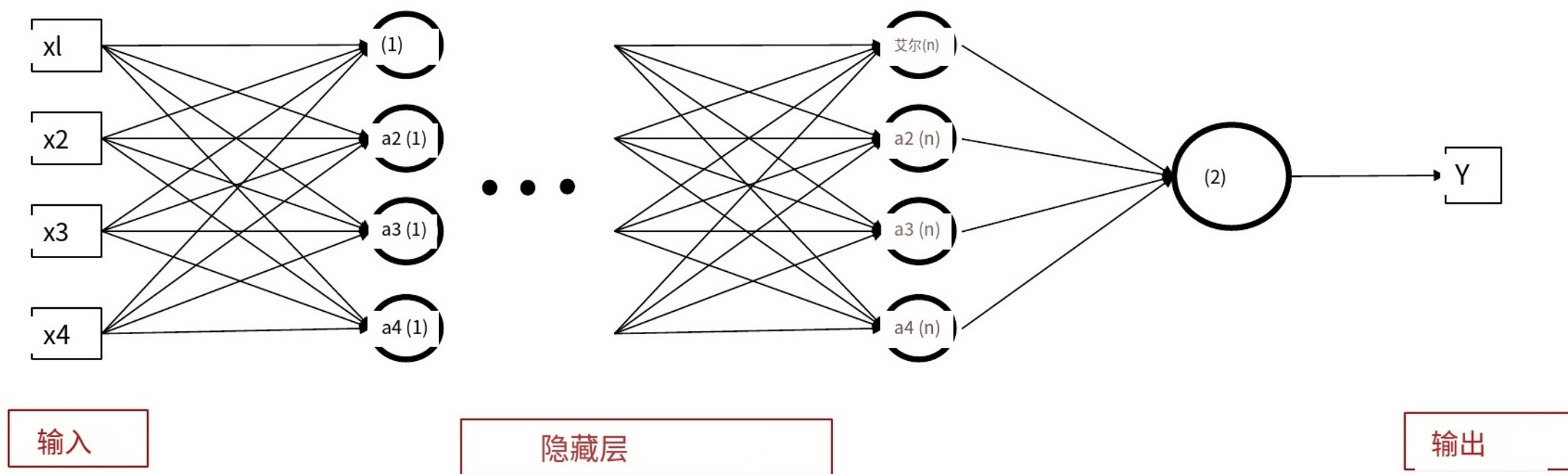


神经网络的普遍逼近性

[霍尼克 1989] 无论激活函数如何，也不论输入空间的维度如何，具有一个隐藏层的神经网络都可以逼近任何连续函数。

深度神经网络

具有 N 个隐藏层的深度神经网络



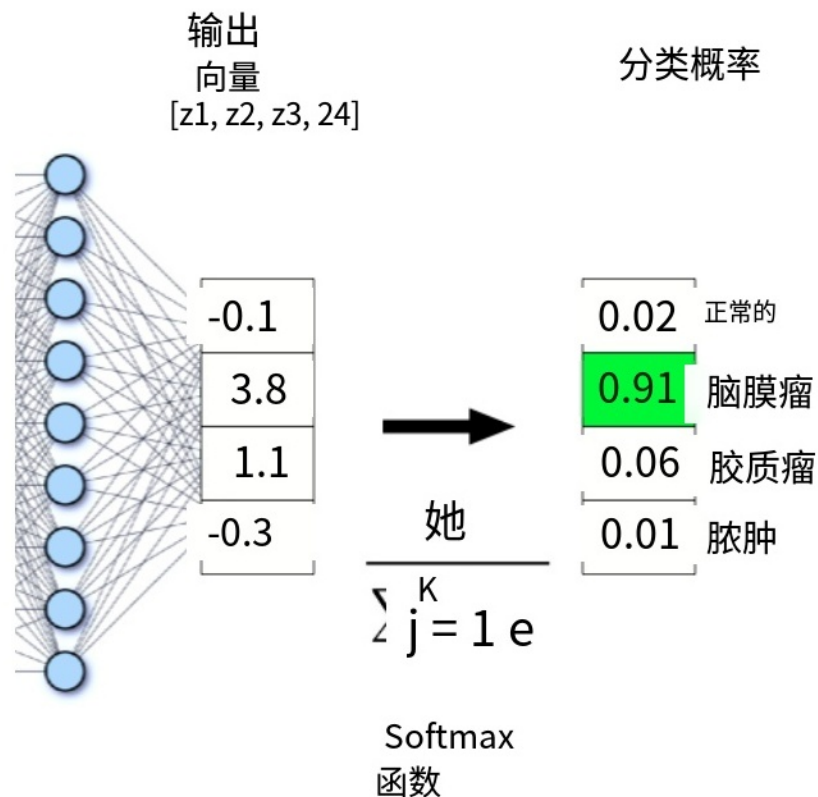
多类别分类中的 Softmax 函数

1. 概率是非负的

通过指数函数 $\exp$ 将 $ZI \in (-\infty, +\infty)$ 转换为非负值

2. 概率之和应为 1 ($\sum I P Y = I | X = 1$) (通过归一化实现)。

$$p(y = i | x) = f(x)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

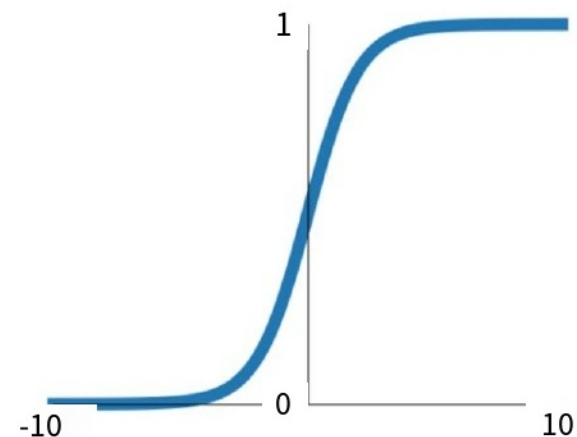


激活函数

Sigmoid: $f(x) = \frac{1}{1+\exp(-x)}$

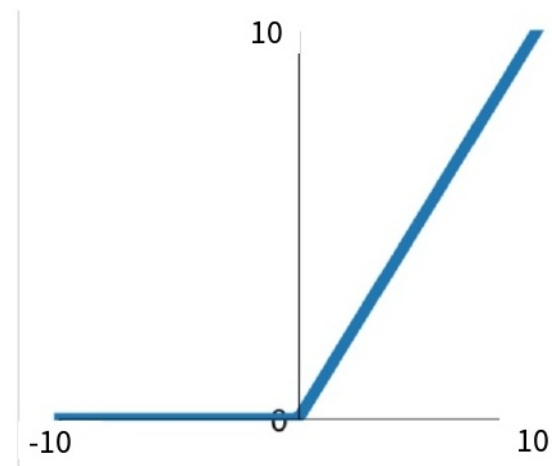
主要用于二分类

- 如今在隐藏层中不常使用。



ReLU: $f(x) = \max(0, x)$

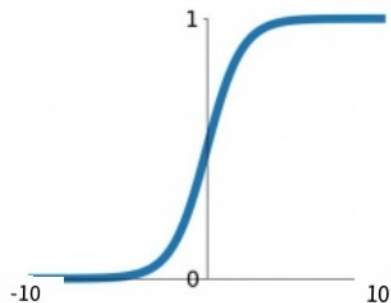
- 隐藏层的良好默认选择



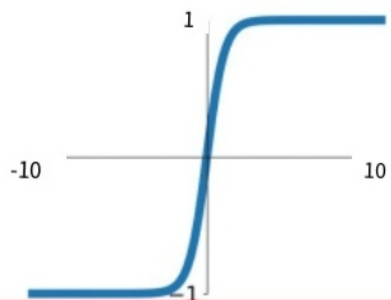
激活函数

ReLU 对于大多数问题来说都是一个很好的默认选择!

Sigmoid $\sigma(x)$
$$\sigma(x) = \frac{1}{1+e^{-x}}$$

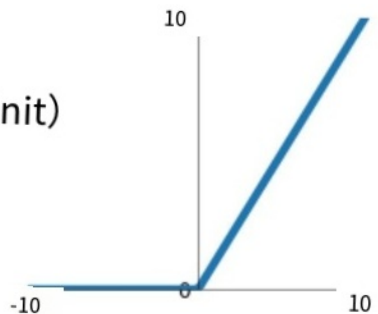


腥臭(x)

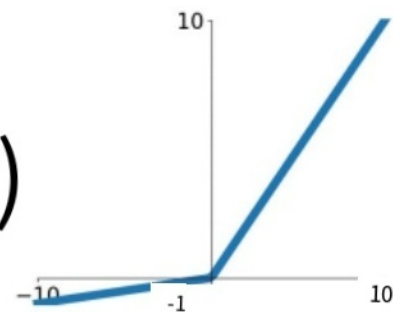


线性整流函数 (Rectified Linear Unit)

马克斯(0,x)



漏水的ReLU
马克斯(0.1 x, x)

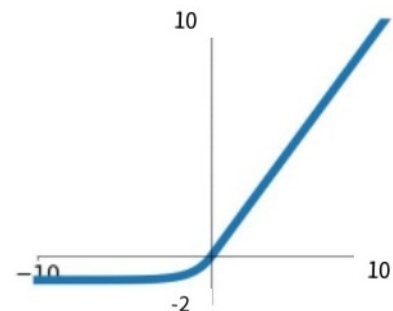


Maxout

Max (wfx + b1, wx + b2)

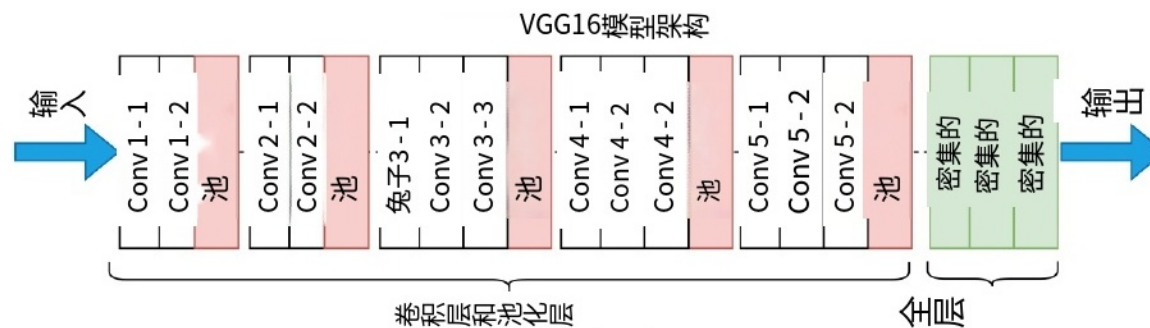
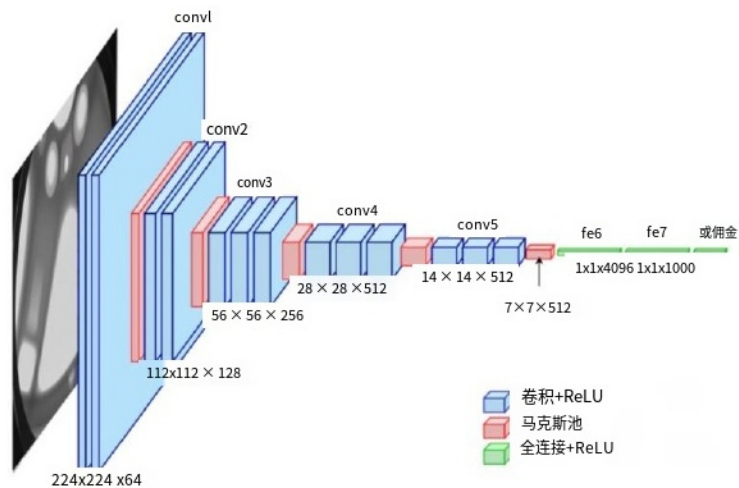
生命

$$\begin{cases} x & x \geq 0 \\ \alpha (e - 1) & x < 0 \end{cases}$$



更深还是更广？

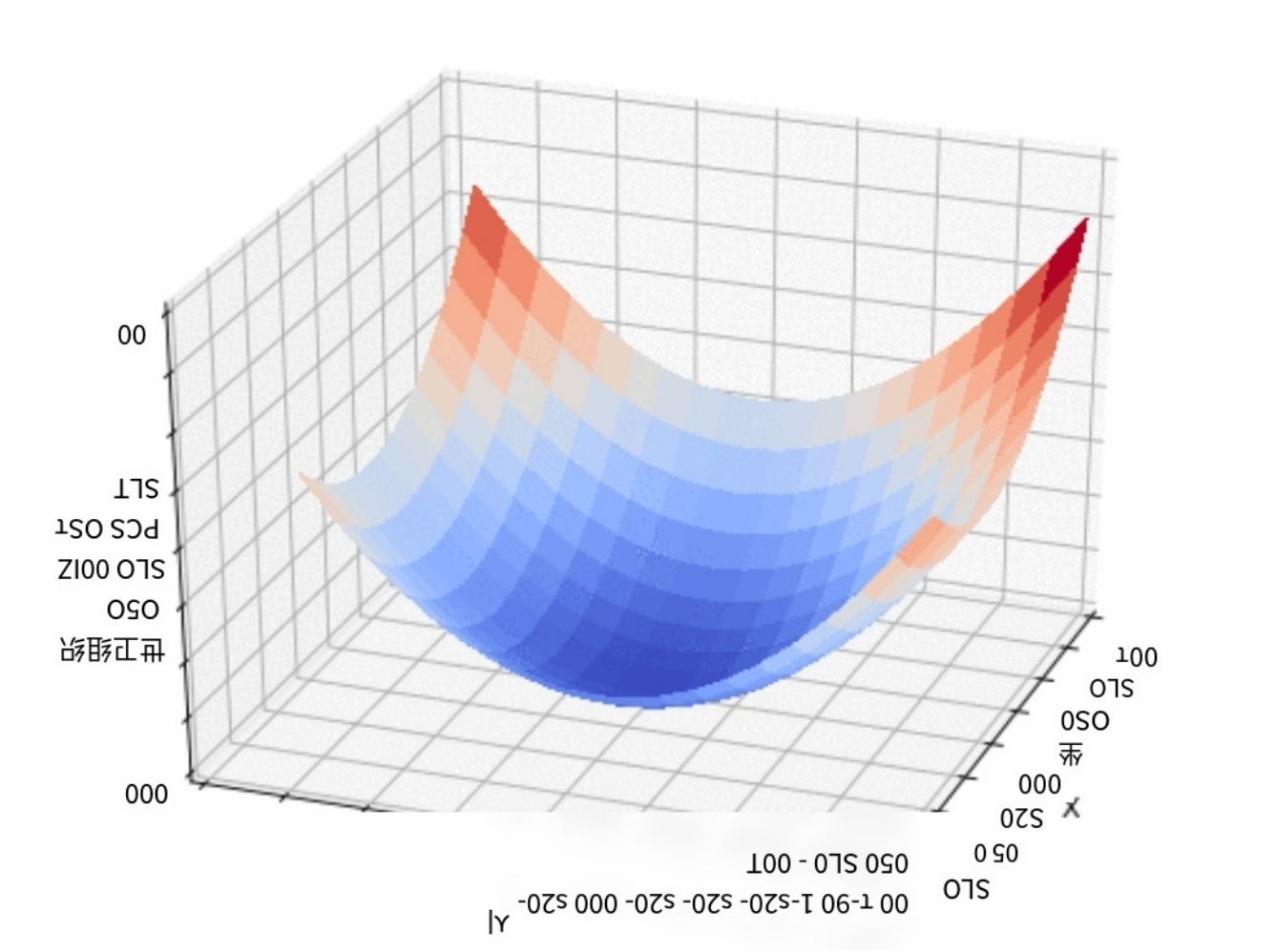
早期的研究人员关注神经网络的宽度
但如今，我们已步入深度学习时代



VGG 是一种经典的深度卷积神经网络

优化

梯度下降



思路：沿着导致损失减少最迅速的方向前进。

$$\mathbf{W} \leftarrow \mathbf{W} - \lambda \frac{\partial L}{\partial \mathbf{W}}$$

关于货物

关于参数

梯度下降

$$\mathbf{W} \leftarrow \mathbf{W} - \lambda \frac{\partial L}{\partial \mathbf{W}} = \mathbf{W} - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^N L_i}{\partial \mathbf{W}} = \mathbf{W} - \lambda \frac{1}{N} \sum_{i=1}^N \frac{\partial L_i}{\partial \mathbf{W}}$$

$$L_i = - \sum_{j=1}^K y_j \log(p_j)$$

· K 表示类别数量，y 是一个独热编码向量

将预测分数转换为概率



概率必须为 ≥ 0

概率之和必须为1

猫	3.2	经验值	24.5	正常化	0.13
车	5.1		164.0		0.87
青蛙	-1.7		0.18		0.00

Softmax函数

$$s = f(x_i, W)$$

这是

埃斯克

Σ 吗



损失函数

比较两个概率分布：
交叉熵

$$L = -\frac{1}{N} \sum_i p_i \log(q_i) = -\frac{1}{N} \sum_i \log \left(\frac{e^{s_{yi}}}{\sum_j e^{s_j}} \right)$$



概率必须大
于等于 0

概率之和必须
为 1

猫	3.2	指数	24.5	标准化	0.13	比较
汽车	5.1		164.0		0.87	
青蛙	-1.7		0.18		0.00	

1
0
0

$s = f(x(W))$

是

$\frac{e^{s_k}}{\sum_j e^{s_j}}$

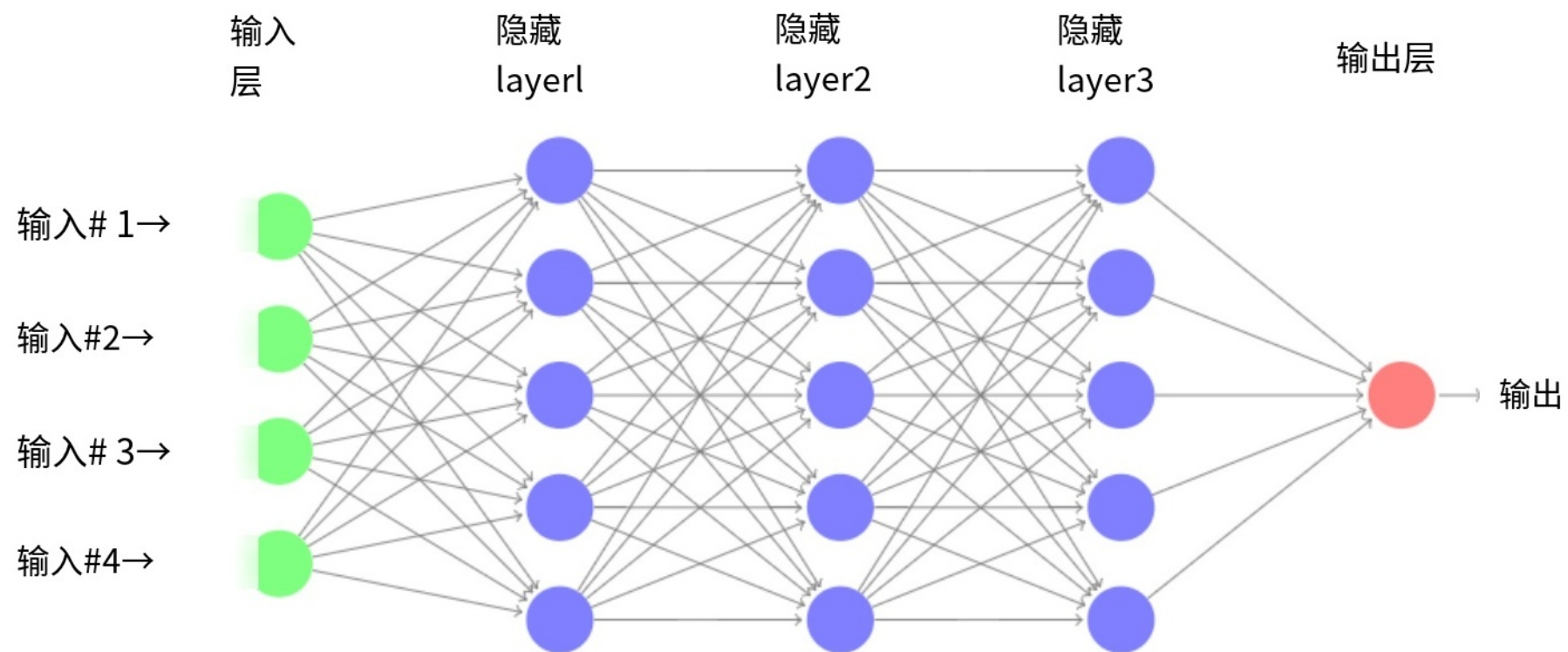
我们如何得到 W 和 B 的值?

- 定义一个损失函数

- Softmax 分类器

优化权重 W 和偏置 B 以最小化损失函数

反向传播：计算梯度



计算图

前进

分解为基本操作

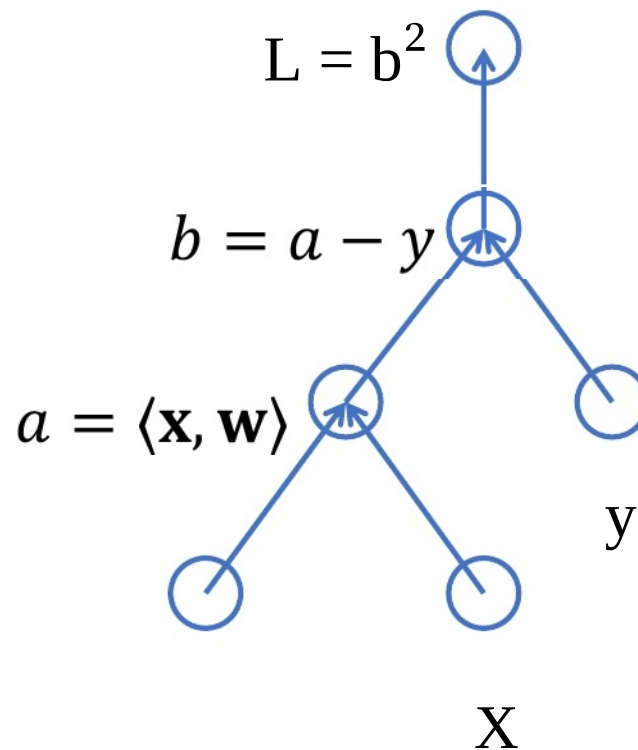
构建一个有向无环图以
展示计算过程

向后的

通过链式法则求梯度

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \cdots \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x}$$

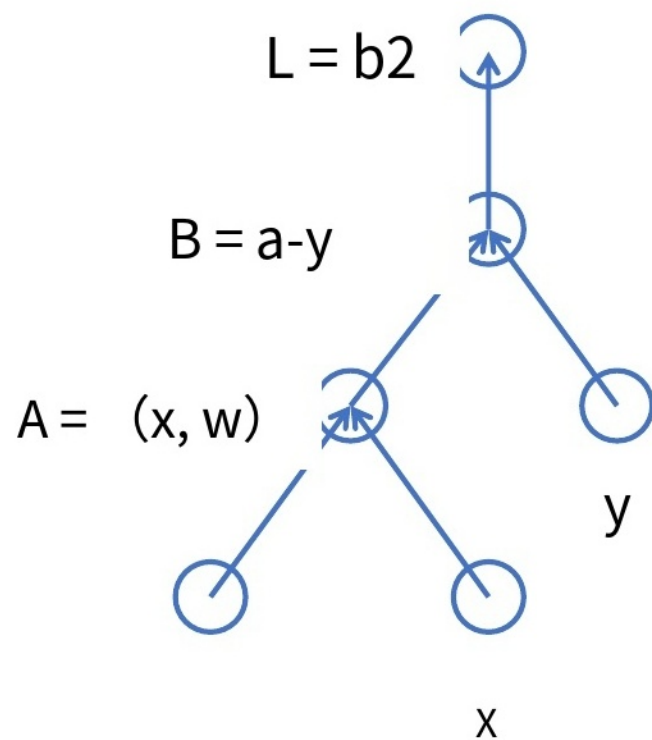
假设 $L = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



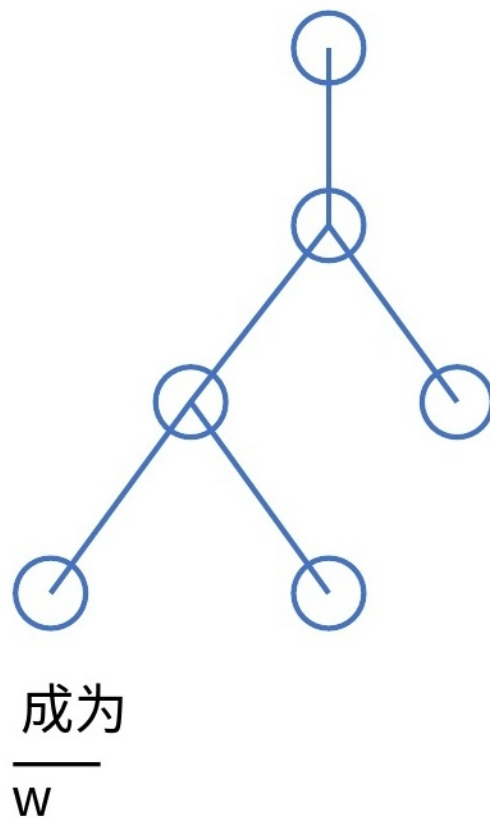
反向累加

假设 $L = (x, w > -y)^2$

向前

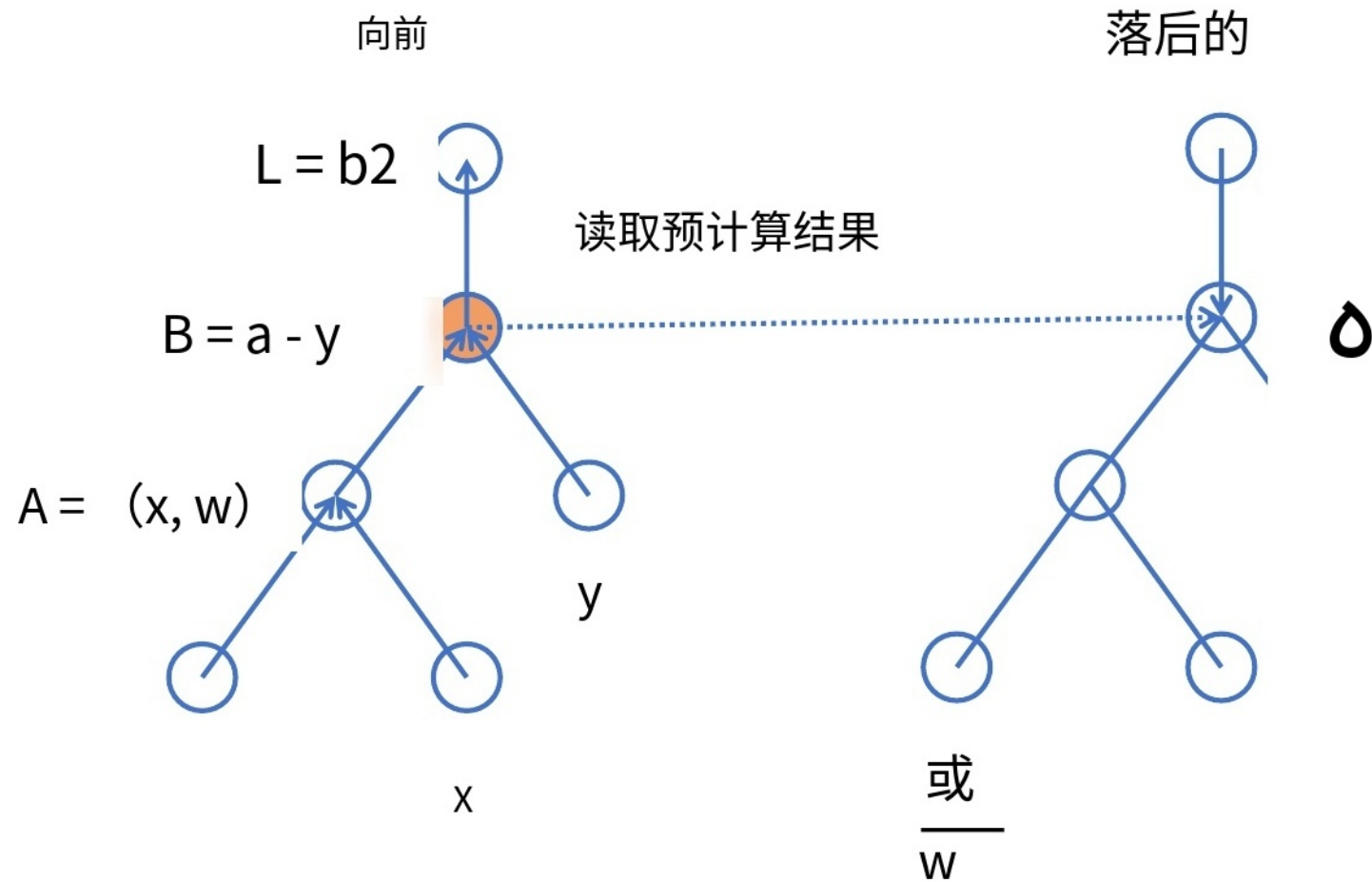


落后的



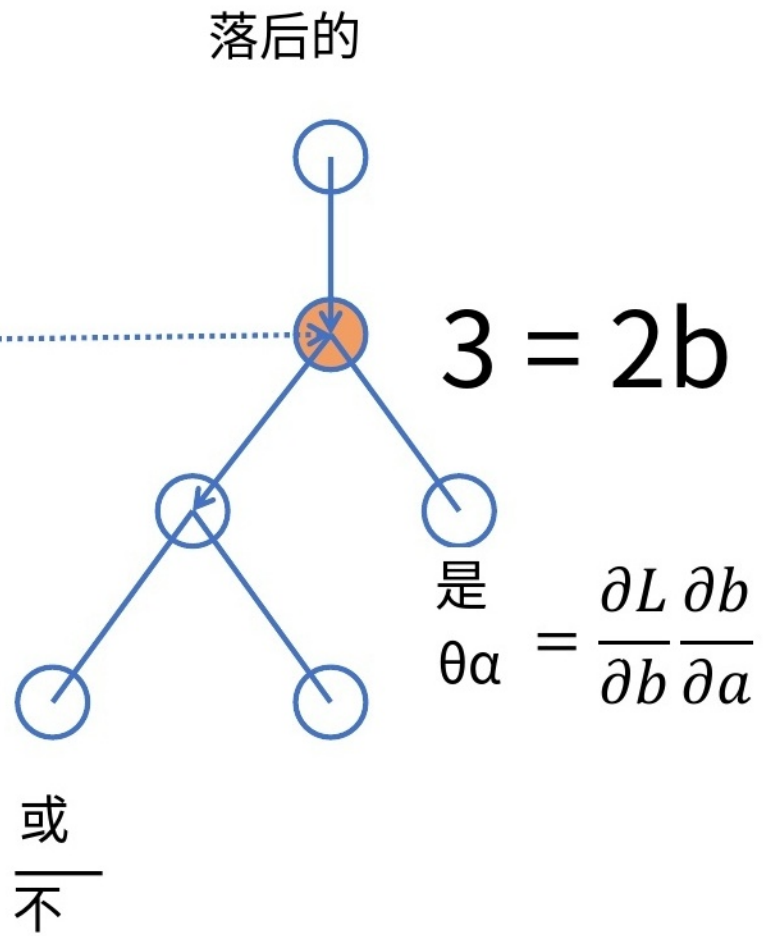
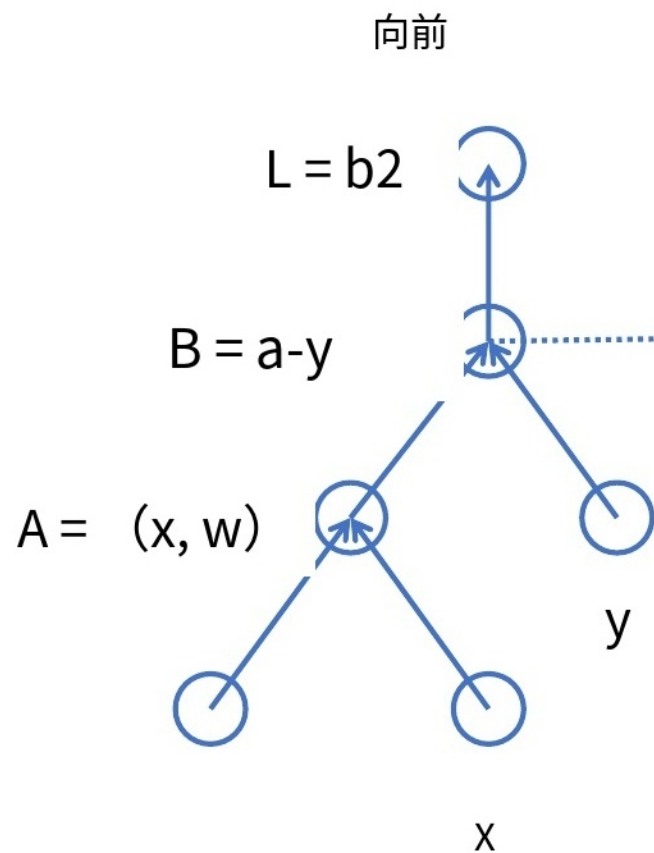
反向累加

假设 $L = (x, w) - y)^2$



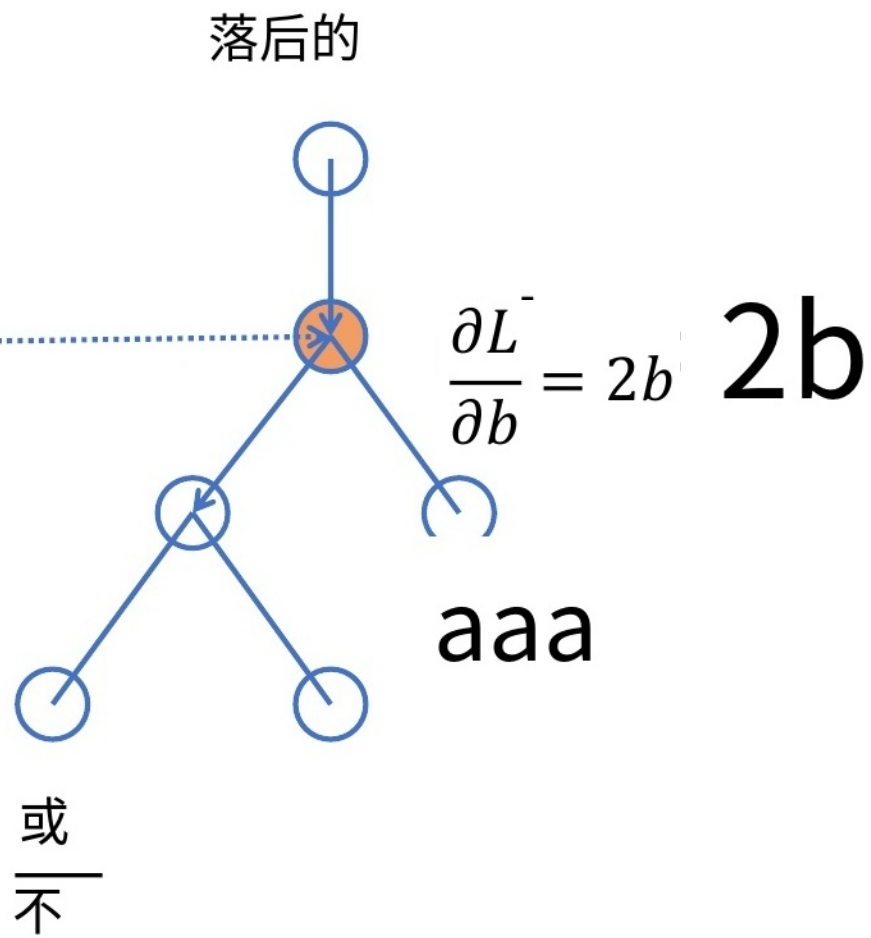
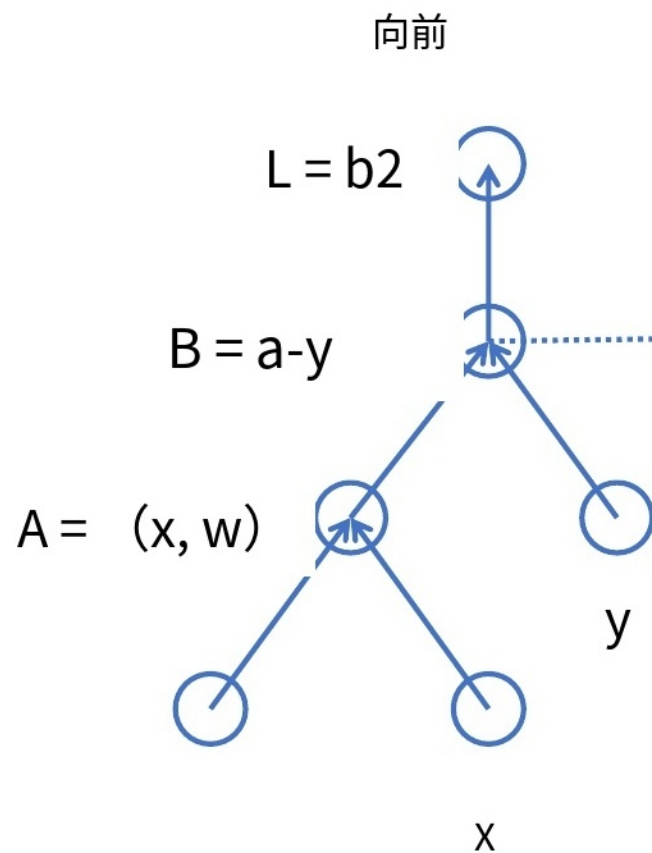
反向累加

假设 $L = (x, w) - y)^2$



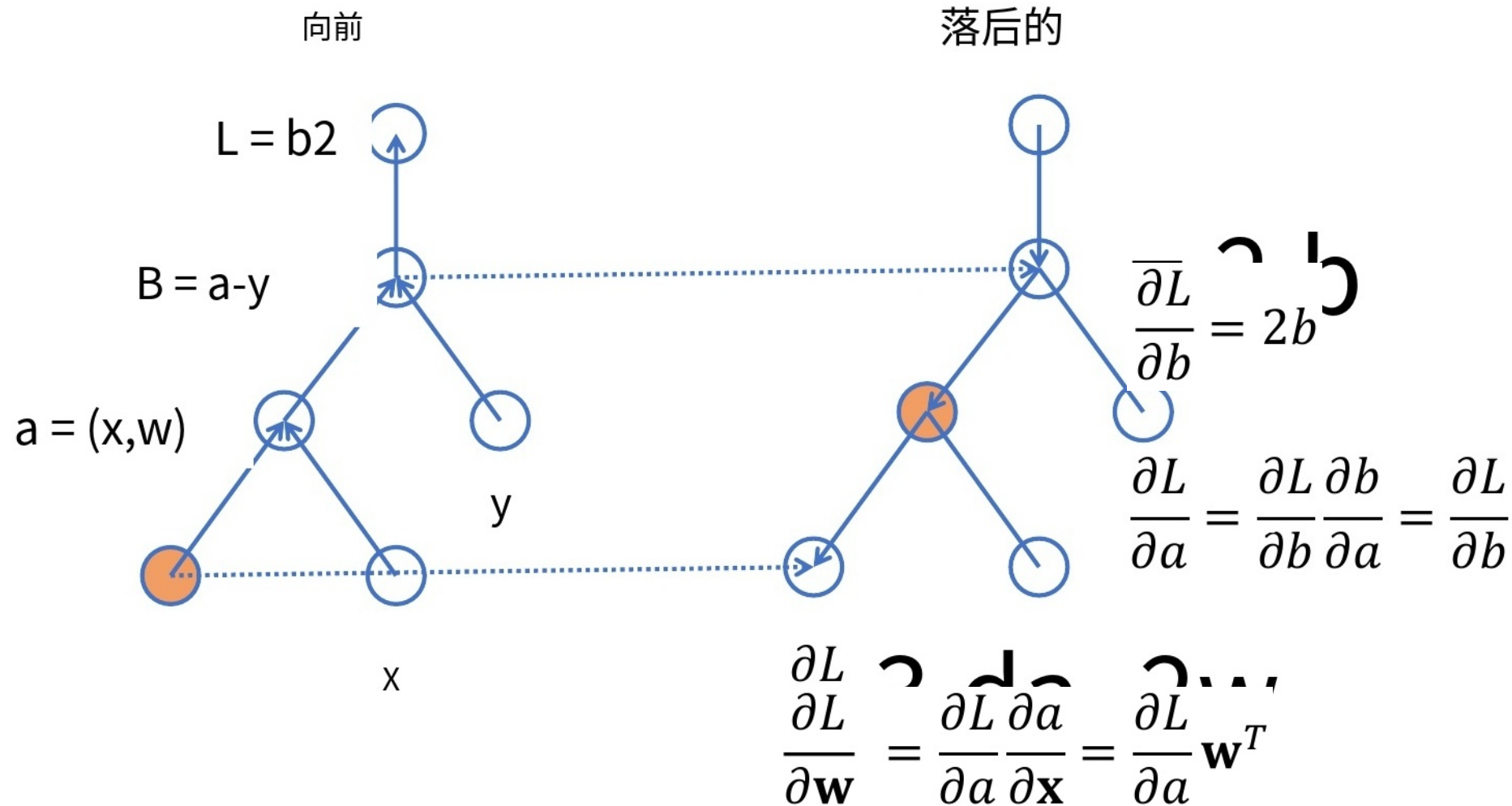
反向累加

假设 $L = (x, w) - y)^2$



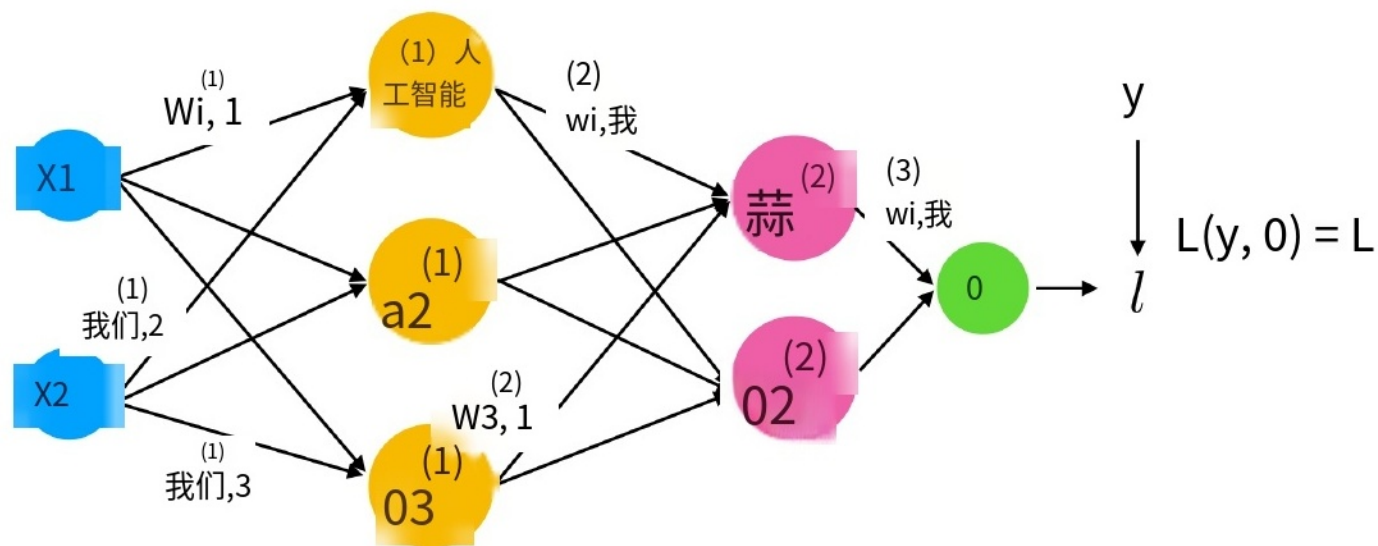
反向累加

假设 $L = (x, w) - y)^2$



深度神经网络难以优化！

梯度消失问题



假设应用了 S 型激活函数：

$$\frac{\partial l}{\partial w_{1,1}^{(1)}} = \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}}$$

$$a_1^{(2)} = \sigma(z_1^{(2)}) = \frac{1}{1 + \exp(-z_1^{(2)})}$$

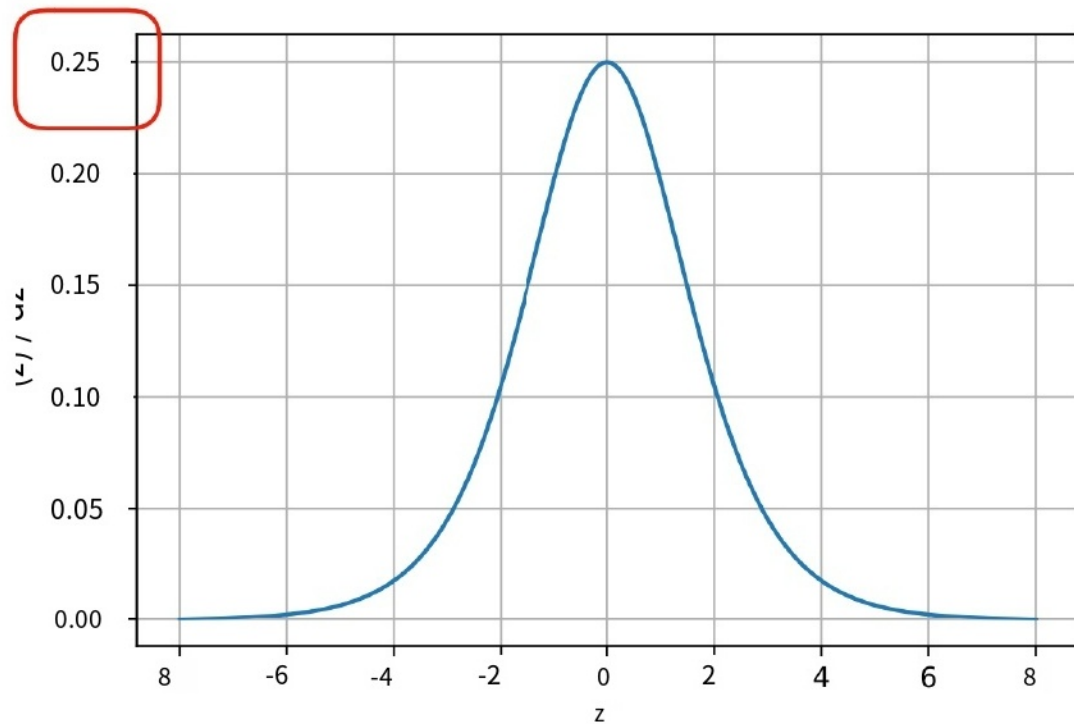
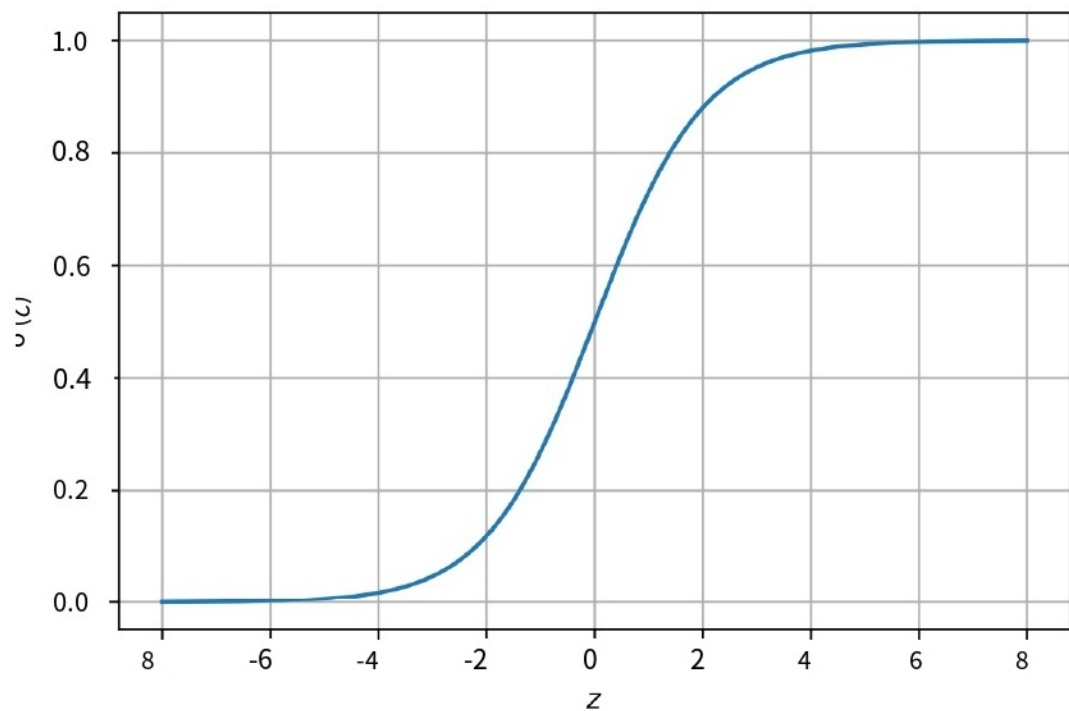
$$z_1^{(2)} = w_{1,1}^{(2)} a_1^{(1)} + w_{2,1}^{(2)} a_2^{(1)} + w_{3,1}^{(2)} a_3^{(1)}$$

梯度消失问题

S 型函数的最大梯度为 **0.25**。

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$



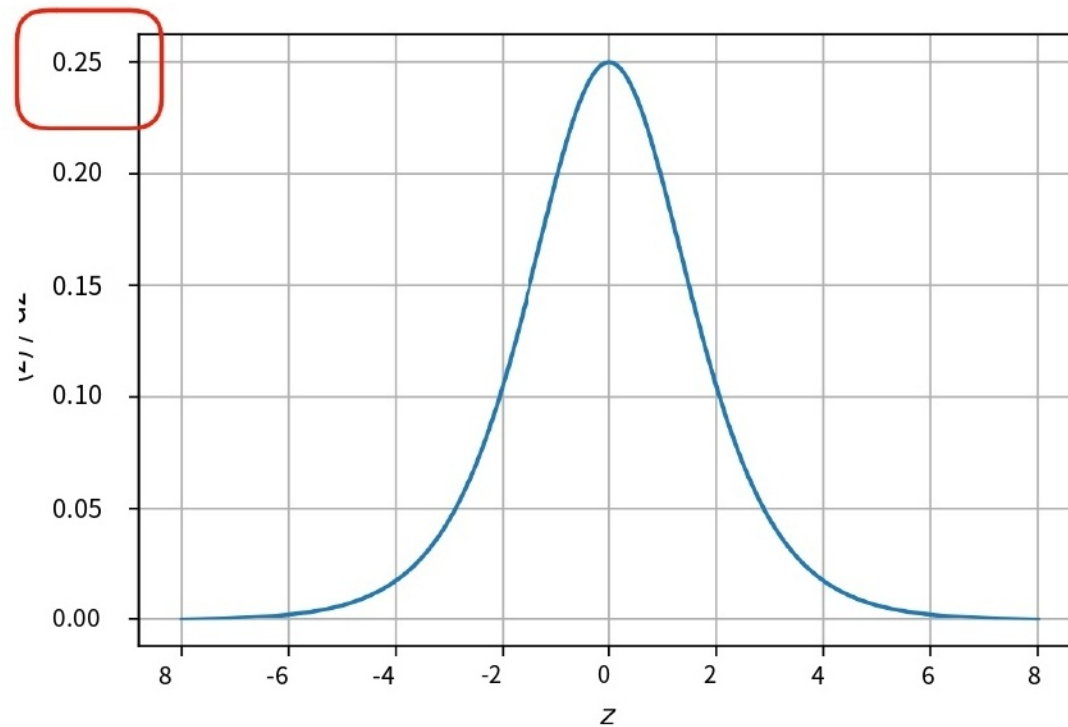
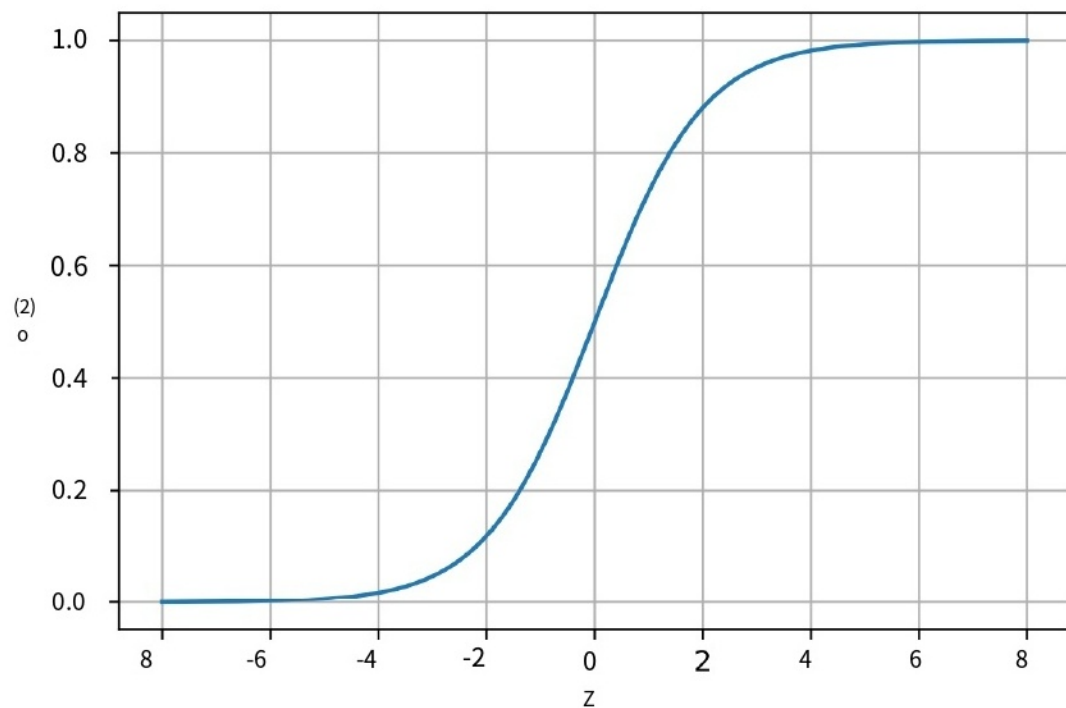
梯度消失问题

S 型函数的最大梯度为 **0.25**。

假设使用 10 层 S 型函数: $0.25^{10} \approx 10^{-6}$

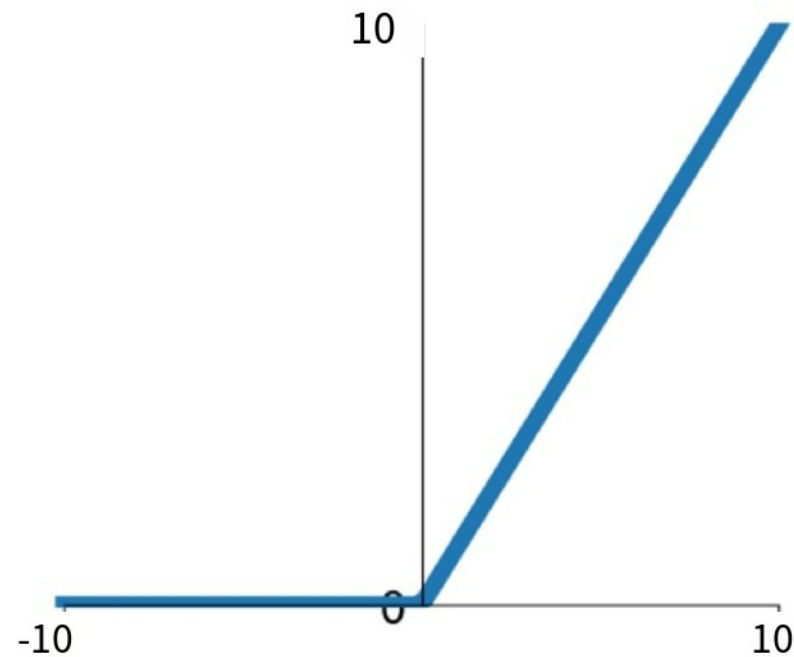
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$



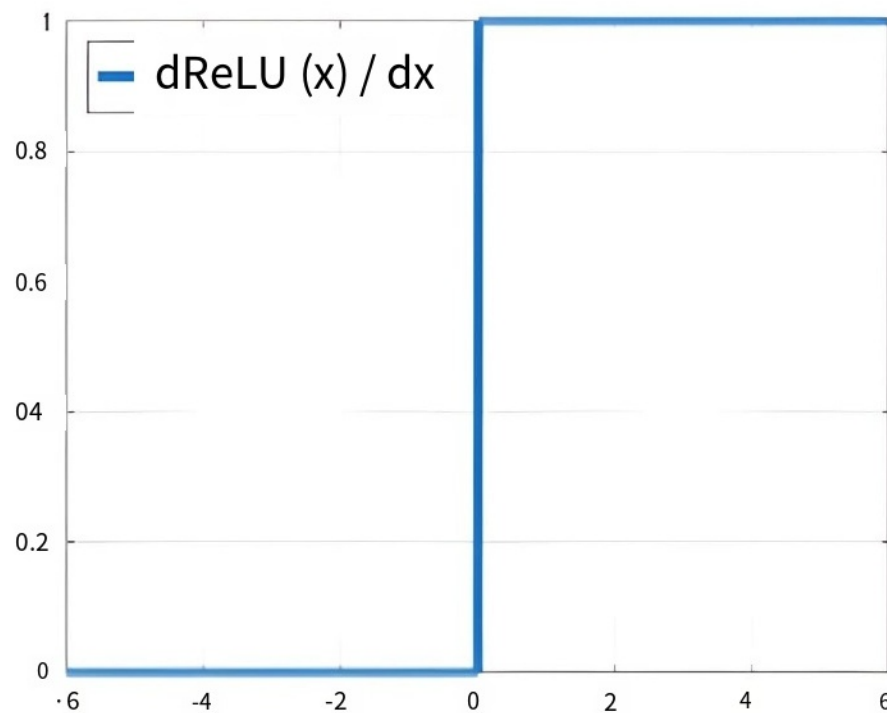
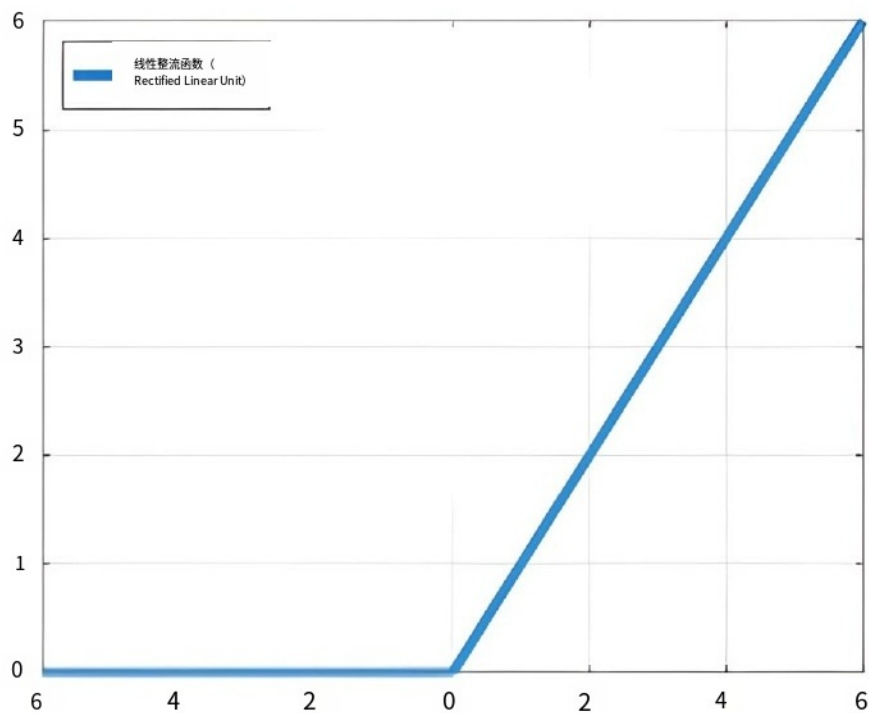
ReLU 怎么样?

ReLU: $f(X) = \max(0, X)$



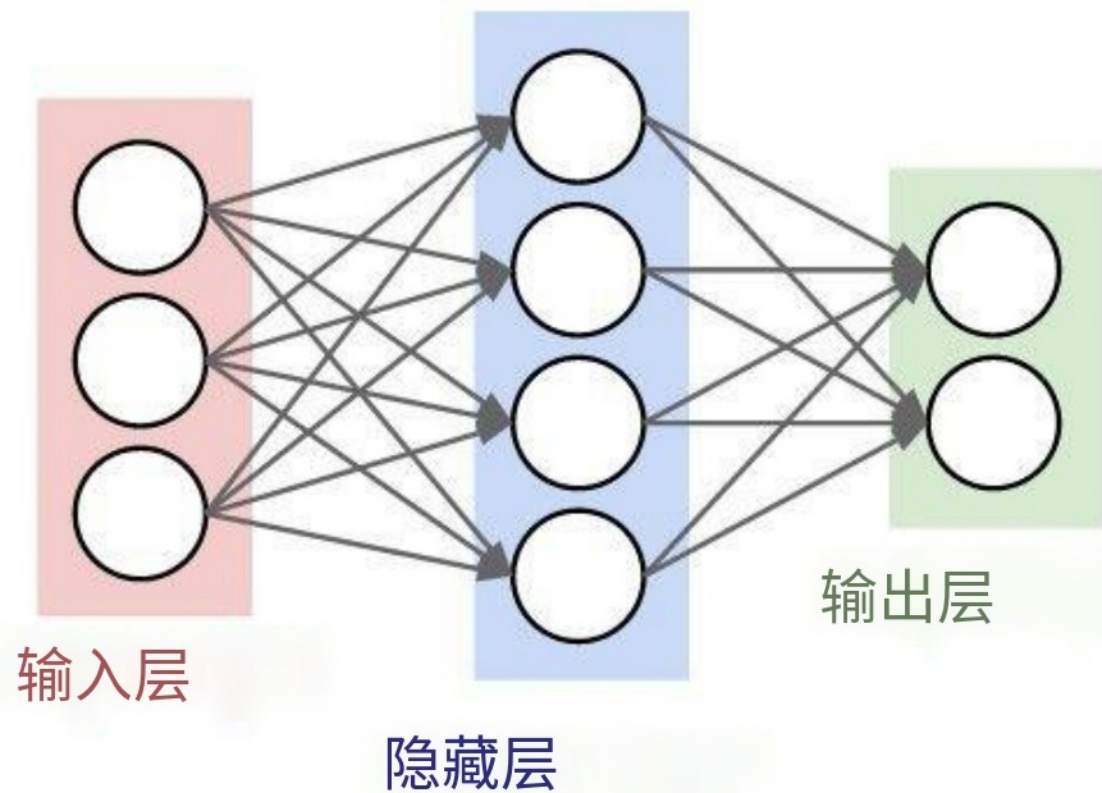
ReLU 没有梯度消失问题

ReLU: $f(X) = \max(0, X)$



权重初始化

- 问：使用 w =常量初始化时会发生什么？



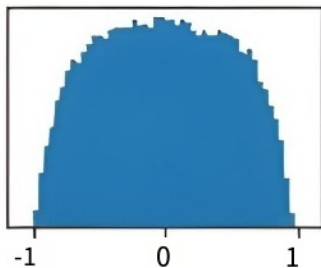
所有神经元都以**相同的方式**进行更新，这使得它们无法学习不同的内容。

具有相同方差的随机初始化

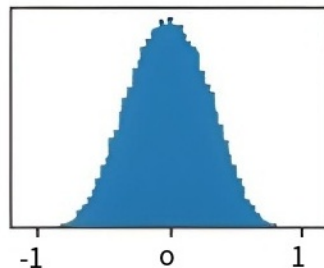
```
dims = [4096] * 7      Forward pass for a 6-layer
hs = []                net with hidden size 4096
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

所有深度网络层的激活值都趋于零

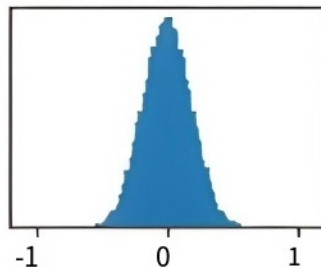
第一层均值
=-0.00, std=0.49



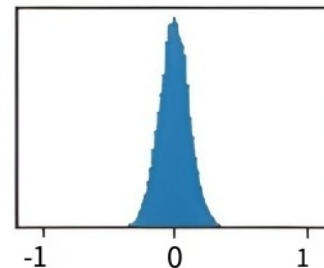
第二层均值
=0.00 std=0.29



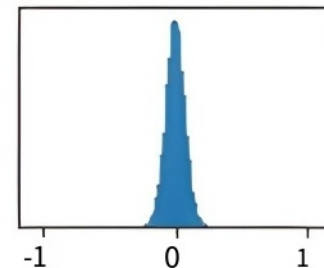
第三层均值
=0.00 std=0.18



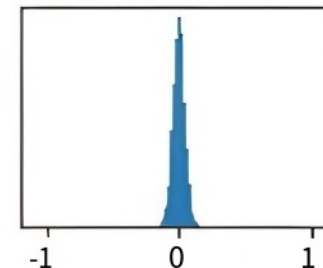
第四层均值
=-0.00 std=0.11



第五层均值
=-0.00 std=0.07



第六层均值
=0.00 std=0.05



具有相同方差的随机初始化

```
dims = [4096] * 7      Forward pass for a 6-layer
                        net with hidden size 4096
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

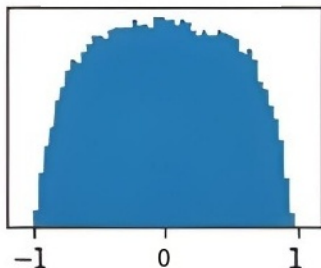
所有深度网络层的激活值
都趋于零

关于梯度呢？

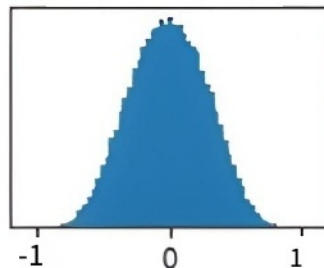
$$x_{n+1} = \tanh(z_{n+1}) \quad z_{n+1} = w_n x_n + b$$

$$\frac{dx_{n+1}}{dw_n} = \frac{dx_{n+1}}{dz_{n+1}} \cdot x_n \approx 0$$

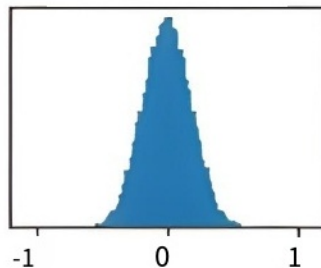
Layer 1
mean=-0.00
std=0.49



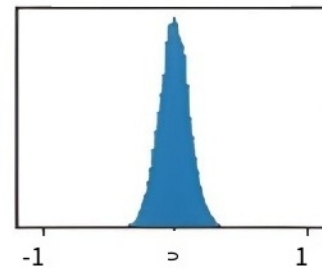
第二层均值
=0.00 std=0.29



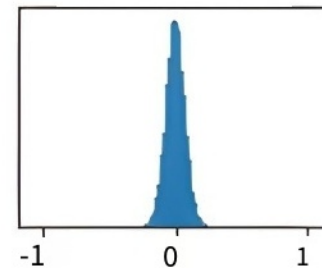
第3层均值
=0.00 std=0.18



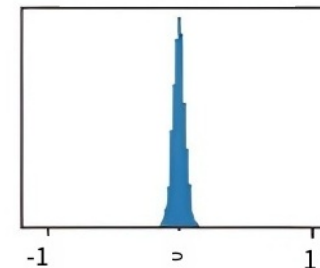
第4层均值
=-0.00 std=0.11



5层
意思是std = 0.07
= -0.00



6层
意思是std =
0.05 = 0.00



泽维尔初始化

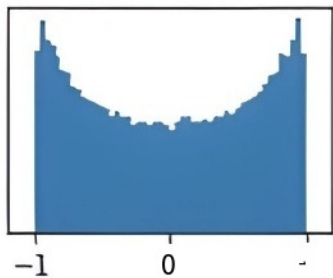
```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” 初始化:

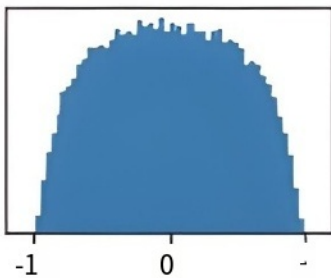
$$std = \frac{1}{\sqrt{D_{in}}}$$

每一层的输出方差 $\approx$ 输入方差

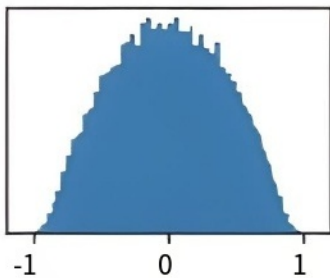
第一层均值
=-0.00, std=0.63



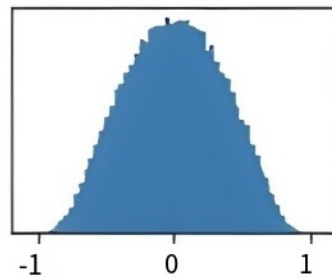
第二层均值
=-0.00 std=0.49



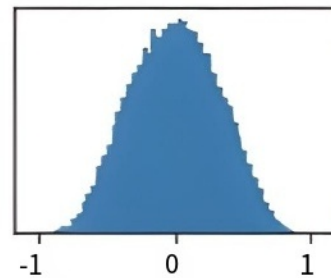
第三层均值
=0.00 std=0.41



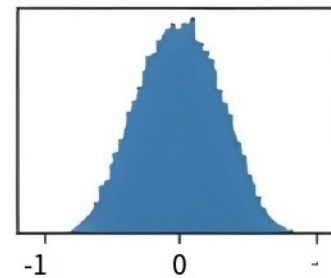
第四层均值
=0.00 std=0.36



第五层均值
=0.00 std=0.32



第六层均值
=-0.00 std=0.30



凯明初始化

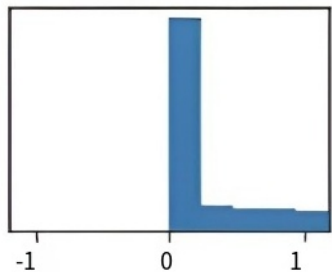
· 专为 ReLU 激活函数设计

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

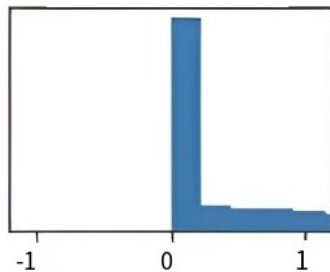
凯明初始化:

$$std = \sqrt{\frac{2}{D_{in}}}$$

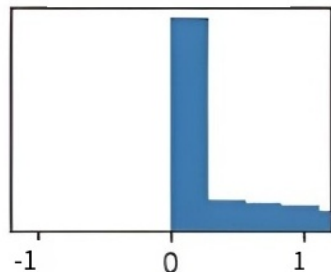
第一层平均=0.57
，标准=0.83



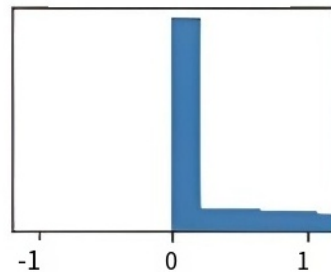
第2层均值
=0.57 std=0.83



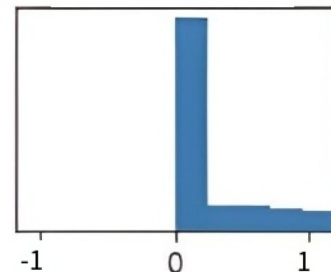
第3层均值
=0.56 std=0.83



第4层均值
=0.55 std=0.81



第5层均值
=0.55 std=0.81



第6层均值
=0.55 std=0.81

